武漢大學
WUHAN UNIVERSITY

《分布式软件原理与技术》
第二讲 高并发设计（1）

计算机学院 贾向阳

武漢大學
WUHAN UNIVERSITY

1、 什么是高并发?

高并发

- 高并发是分布式软件的核心目标与挑战之一
- 核心定义
  - 高并发是指系统能够同时处理大量并发请求的能力
  - 通常以吞吐量（QPS/TPS）和并发用户数为衡量指标。
- 通俗理解
  - 就是系统“扛得住”多少流量。
  - 例如，双十一零点瞬间百万级请求涌入，系统不能崩溃。

并发评估指标

指标	全称	定义	示例
QPS	Queries Per Second	每秒查询数，常用于读场景	搜索引擎每秒处理多少搜索请求
TPS	Transactions Per Second	每秒事务数，常用于写场景	下单、支付等事务操作
并发用户数	Concurrent Users	同一时刻正在操作的活跃用户数	在线聊天室的同时在线人数

注意：高并发≠高吞吐，还需关注响应时间，否则只是“慢吞吞地处理大量请求”。

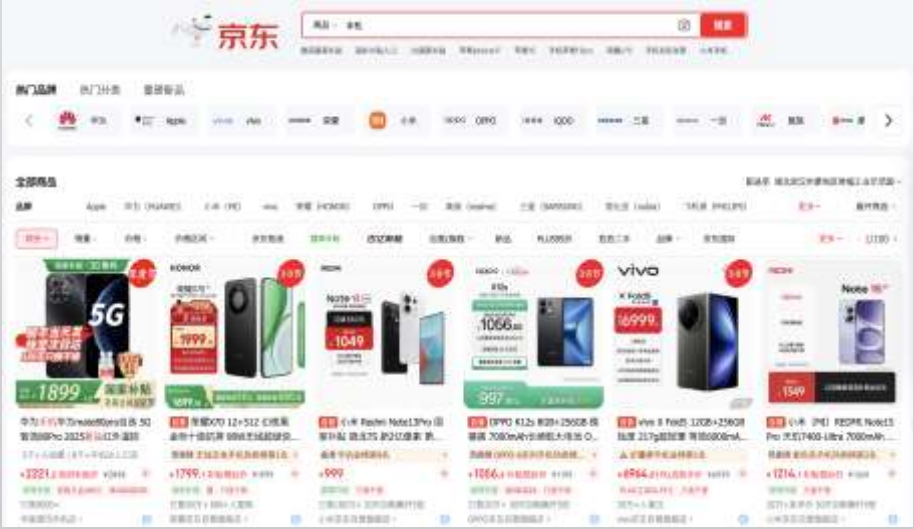
高并发与其他属性关系

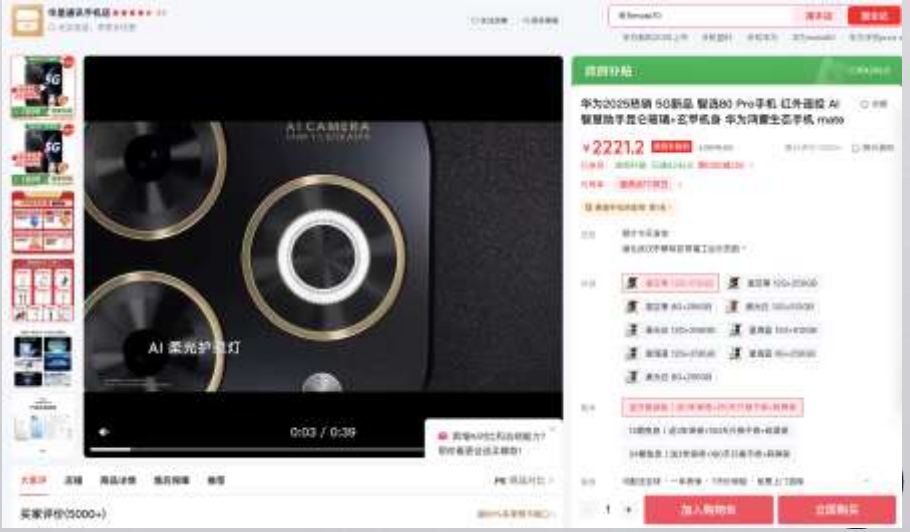
- 依赖高性能：单个请求处理慢，并发能力必然受限。
- 依赖高可用：服务不可用，并发为0。
- 与高可靠/一致性的权衡：为了保证强一致和高可靠，可能采用同步复制、分布式事务等，这会增加延迟，降低并发能力。
- 典型案例：秒杀系统——通过异步削峰、最终一致性来保障高并发，牺牲强一致。

2、 高并发问题分类

高并发读系统

- 高并发读系统是指在系统整体请求量中，读操作占据了绝对主导地位，且要求在海量请求下仍能提供快速、稳定的数据访问能力。
- 典型业务场景
  - 搜索引擎：如百度、Google。用户发起搜索请求，系统需从海量索引中快速匹配结果。
  - 电商商品搜索：如淘宝、京东的商品列表页，用户通过关键词筛选和查找商品。
  - 商品详情页：如淘宝、京东的商品详情页，承载了商品最核心的信息展示。





高并发读系统的特征

核心特征：读写之间的巨大鸿沟

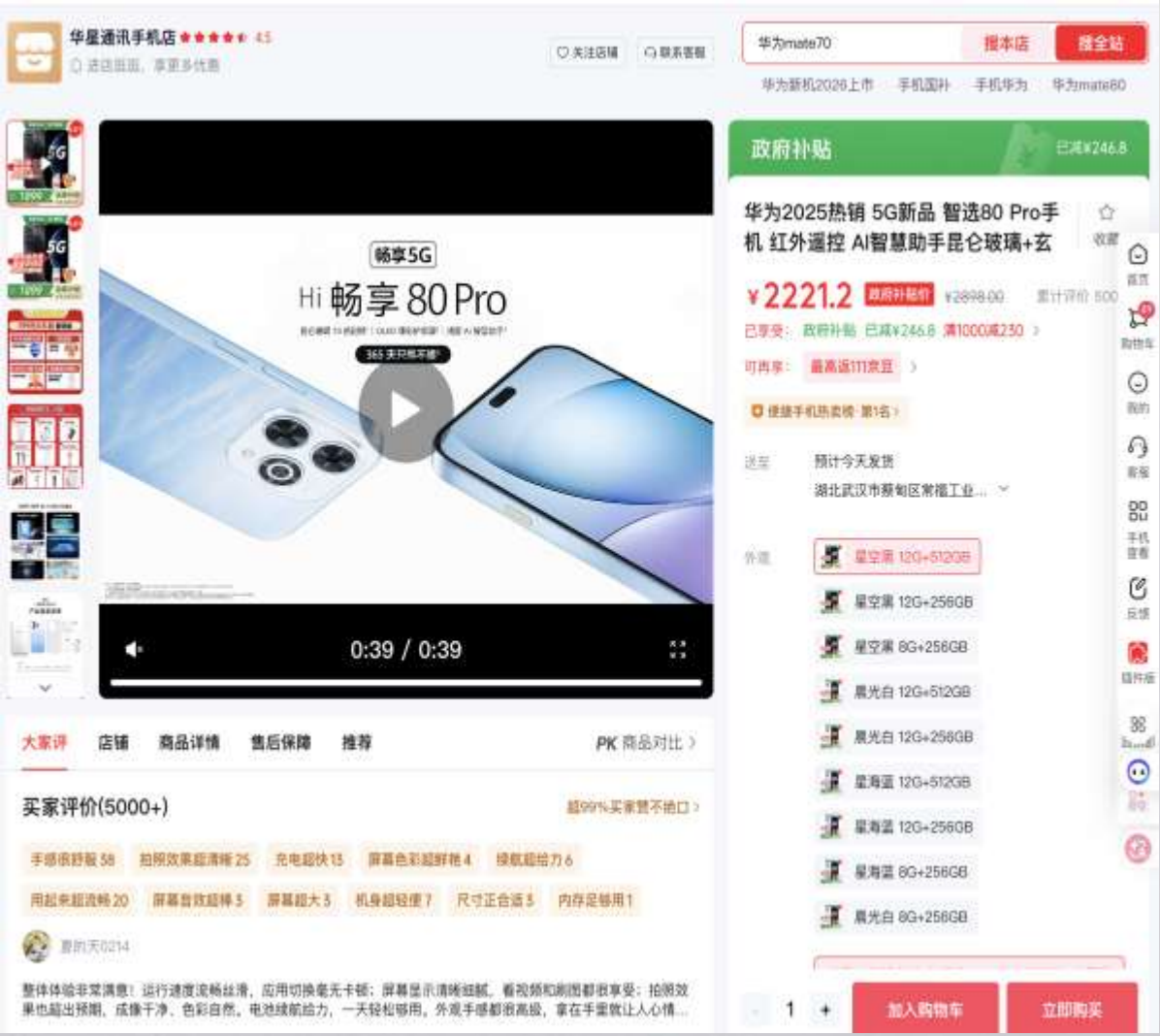
- **1. 请求规模：读的量级远超写**
- 读的一端（C端用户）面向所有互联网用户，规模可达数亿甚至数十亿级。例如，百度日均处理搜索请求超过60亿次。写的一端（内容/商品发布者）面向专业用户或商家，规模通常在百万或千万级。
- **2. 响应时间：读是毫秒级博弈，写可容忍延迟**
- 读的一端要求毫秒级响应。Google研究显示，响应时间超过100ms，用户就会感到轻微延迟；超过1秒，用户流失率大幅上升。
- 写的一端响应时间要求相对宽松，例如，发布一篇博客，可能几分钟甚至几小时后才会被搜索引擎收录并展示给其他用户。
- **3. 频率：读的操作频率远高于写**
- 读的一端用户可能每分钟都在进行搜索、浏览、刷新。例如，在微博热点事件中，用户会持续刷新页面获取最新进展，产生高频读请求。
- 写的一端发布者可能每天甚至每周才发布一篇内容或修改一次商品信息。

武漢大學 WUHAN UNIVERSITY

高并发读系统的挑战

核心挑战

- **流量海啸：**大促或热点事件时，流量可在瞬间飙升至日常的数百倍（如双11零点，热门商品详情请求达数十万至数百万QPS）。
- **响应极度敏感：**商品详情页延迟每增加1秒，用户流失率可能提升10%以上，要求99.9%的请求响应在100ms以内。
- **数据复杂度高：**一个请求背后往往需要聚合来自不同数据源的数十个字段信息（如基础信息、库存、价格、评价等）。
- **可用性要求苛刻：**作为交易链路起点，服务可用性要求近乎100%，任何故障都将直接导致订单流失



高并发读系统的技术方案

总体思路：分层防御与流量治理

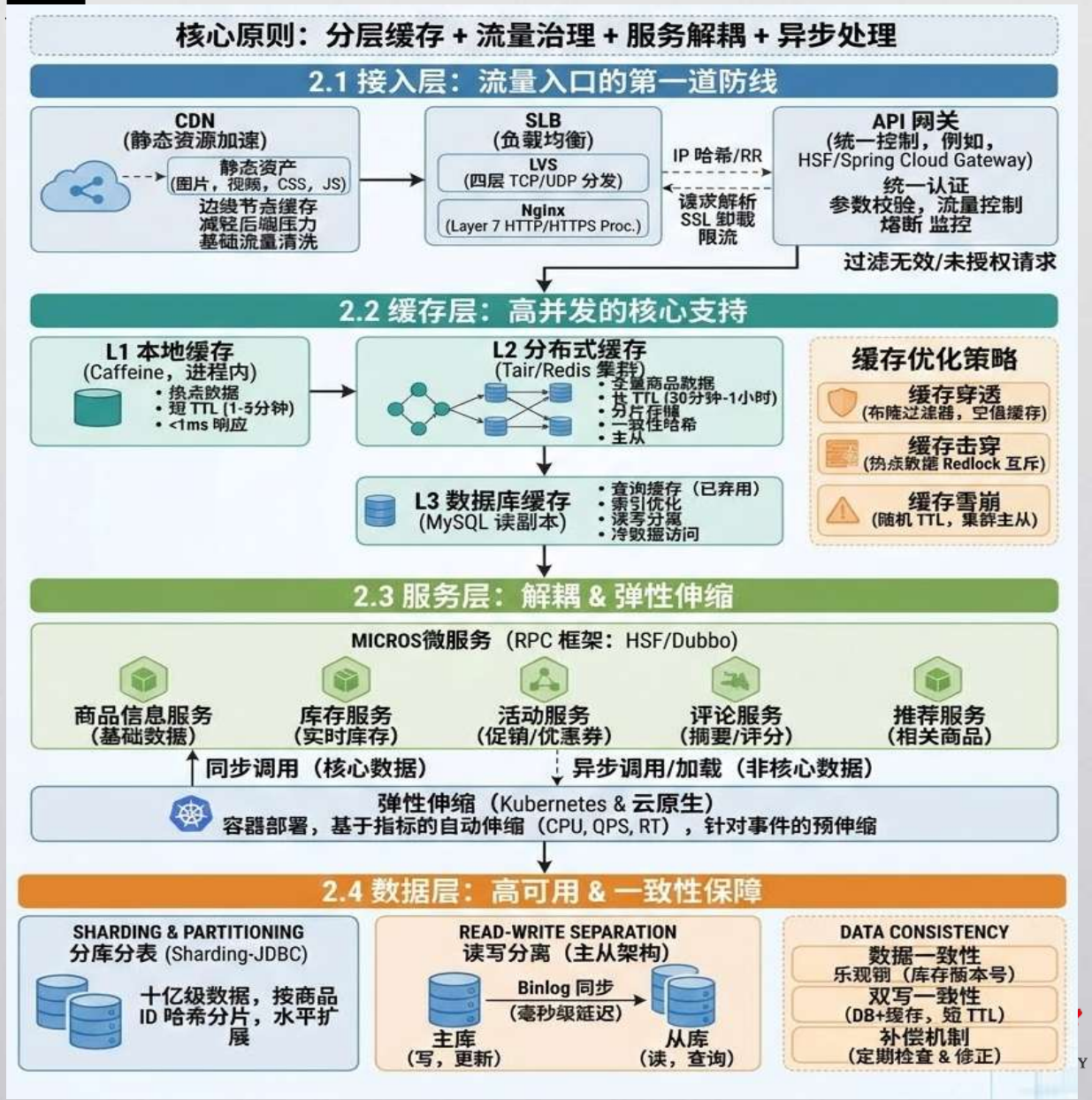
- 面对高并发读，核心策略是“分而治之”与“多层缓冲”，尽量让请求在离用户最近的地方被处理，避免穿透到后端数据库。

特征	接入层	缓存层	服务层	数据层
目标	过滤、分发、隔离流量	降低数据库压力，提升响应速度	解耦与弹性伸缩	高可用性、一致性、实时性
关键技术	CDN加速，SLB负载均衡，API网关	本地缓存，分布式缓存，数据库缓存	服务拆分，RPC调用，弹性伸缩	分库分表，读写分离，数据一致性
优势	过滤无效流量，分发有效流量	快速承接高频热点请求	专岗专责，弹性伸缩	避免单库单表压力过大
挑战	过滤恶意爬虫流量	缓存数据一致性	微服务间的高效调用	数据同步延迟

高并发读系统的技术方案

- 淘宝商品详情API采用的分层防御与流量治理四层架构

https://www.e-com-net.com/article/199983961300224496.htm



高并发写系统

高并发写系统

- 是指在系统整体请求量中，写操作占据了主导地位，且要求在极端流量下仍能保证数据的准确、一致与持久化。这类系统往往是企业的“资金流”和“业务核心”，对数据一致性有严苛要求

典型业务场景

- 广告计费系统（CPM/CPC）：每次广告展示或点击，系统需实时扣减广告主账户余额，要求万级TPS、毫秒级响应、资金零差错。
- 微博/朋友圈发布：亿级用户发布内容，要求快速存储并同步至粉丝 feeds流，同时审核、索引等异步流程需可靠执行。

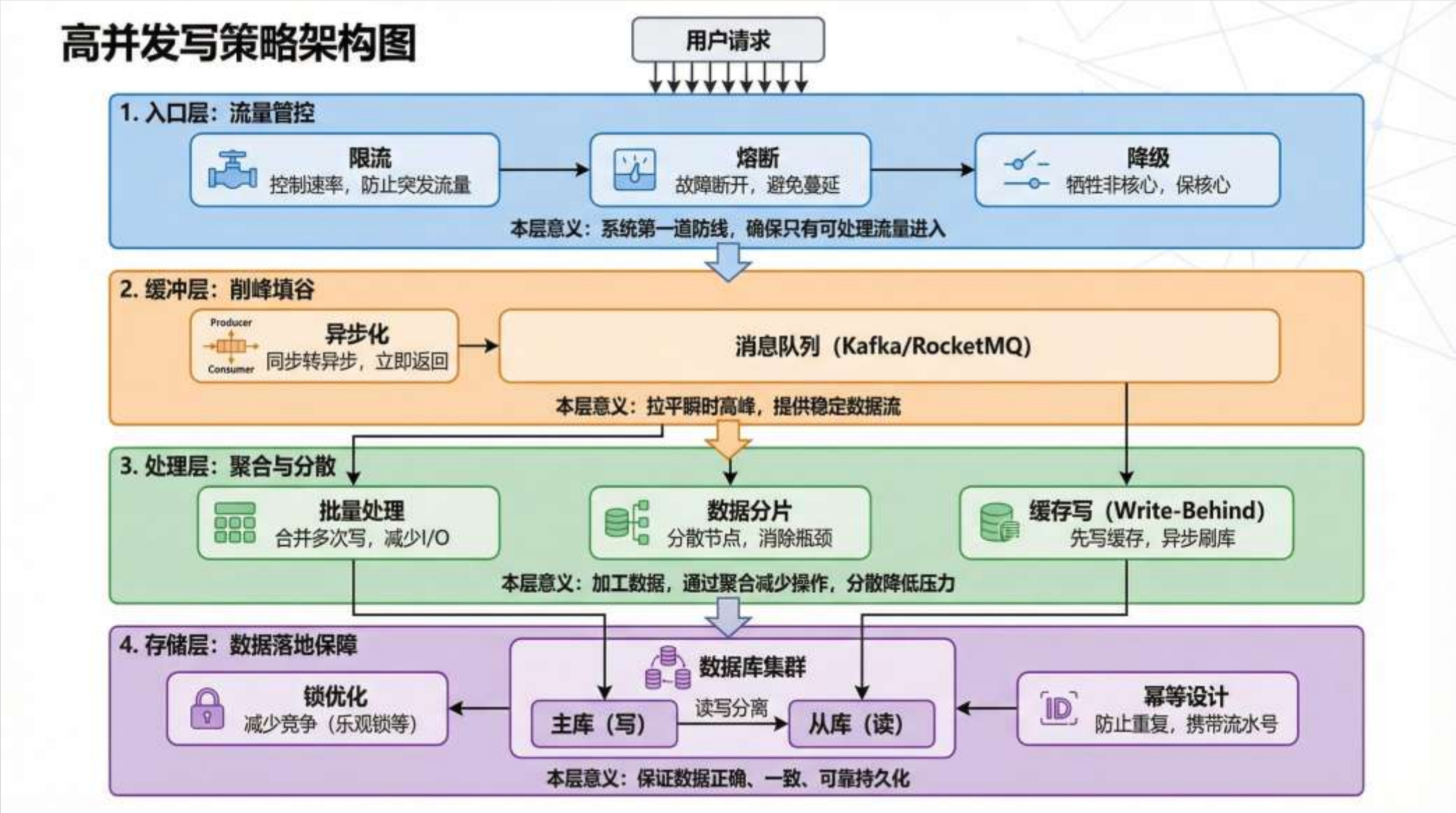
高并发写系统的核心挑战

核心挑战

- **流量洪峰：**360 智慧商业广告计费系统高峰时段TPS可达10万+/秒，朋友圈晚高峰发布量激增。
- **热点行并发：**锁、事务、日志落盘，并发时极易出现行锁竞争和I/O瓶颈。例如广告主账户**余额字段**是典型的热点行，大量并发扣费请求争夺同一行锁。百度凤巢广告系统日均处理数十亿PV（页面浏览），原始日志达20TB/天，需在10分钟内处理1亿条记录
- **强一致性+高性能的矛盾：**传统数据库同步提交机制在高并发下成为瓶颈，需在一致性和吞吐间寻求平衡。
- **数据可追溯性**：所有写操作必须可审计、可对账，需要支持异常补偿。

高并发写系统的技术方案

技术方案



高并发读写混合系统

高并发读写混合系统

- 是指在系统整体请求量中，读操作和写操作同时面临巨大的并发压力，且要求在海量请求下仍能保证数据的实时性、一致性与系统稳定性。这类系统往往处于业务核心链路，对用户体验和资金安全有极高要求。

典型业务场景

- **12306售票系统：**用户查询余票（读）和下单购票（写）同时爆发，2012年春运，系统因瞬时访问量超过14亿次濒临瘫痪。
- **电商秒杀系统：**用户查询库存（高并发读）同时下单扣减库存（高并发写），要求库存数据实时准确。
- **支付系统/微信红包：**用户查询余额（读）同时进行转账/发红包（写），资金类业务对数据一致性要求极高。2017年除夕，微信红包收发量达142亿个，峰值76万/秒.
- **IM/微博/抖音评论系统：**用户发送消息/评论（写）同时接收消息/刷新Feed流（读），要求低延迟、高吞吐

武漢大學 WUHAN UNIVERSITY

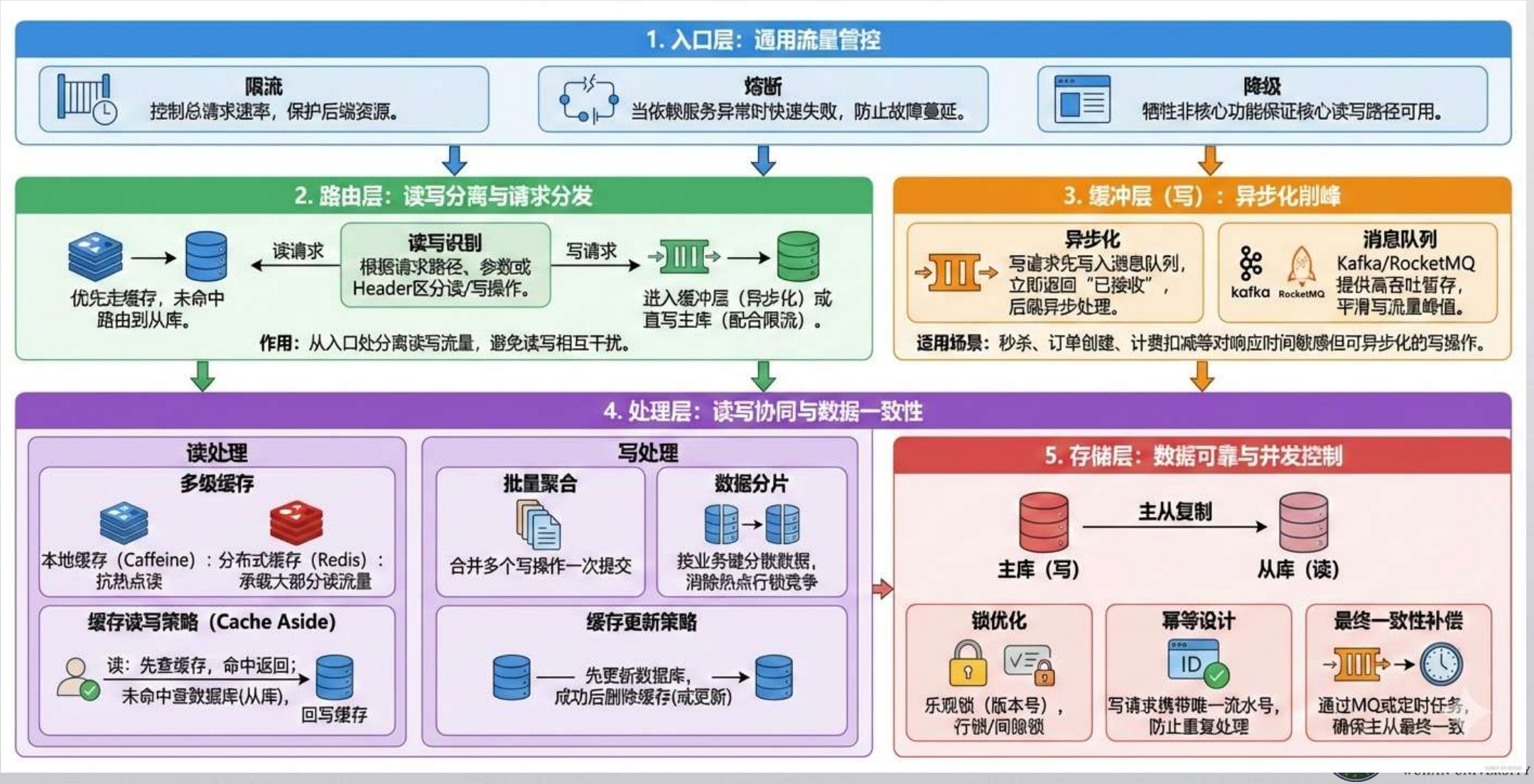
高并发读写混合系统的核心挑战

核心挑战

- **热点数据争用：**当大量请求集中访问同一份数据时（如爆款SKU的库存、春晚红包的同一个红包），该数据所在的分区或节点成为瓶颈.
- **读写互相干扰：**写操作占满磁盘I/O时，读操作被迫等待磁盘响应，导致读延迟飙升。例如，Kafka在开启同步刷盘后吞吐量急剧下降，就是因为磁盘写放大影响了整体性能.
- **强一致性与高性能的平衡：**为了保证数据一致性，需要引入锁或分布式事务，但这些机制会显著增加延迟. 提高并发性能，但可能出现数据不一致（如用户看到库存还有但下单时提示无货）.
- **系统稳定性与可扩展性：**如何在流量洪峰下保持系统稳定，并支持水平扩展

高并发读写混合系统的技术方案

技术方案



3、 动静分离与CDN加速

接入层优化：动静分离与CDN加速

什么是动静分离?

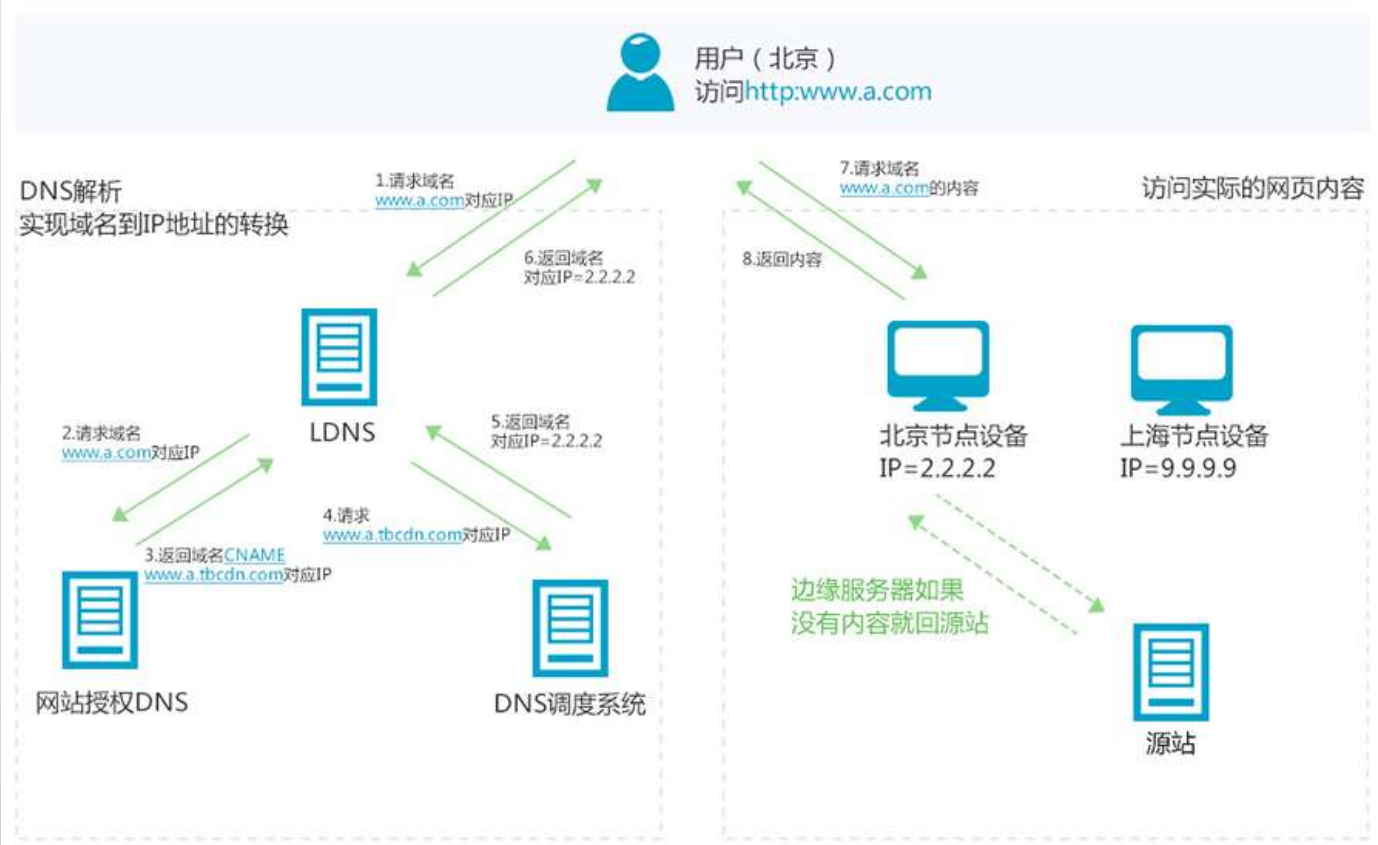
- 将Web应用中的静态资源与动态内容分开处理的一种设计方法
- 前端代码都属于静态资源

类型	定义	典型示例
静态资源	几乎不变或变化频率极低的文件	图片（JPG/PNG）、CSS样式文件、JavaScript脚本、视频、字体、HTML文件
动态内容	服务器代码实时生成的内容	用户个人信息、实时库存、搜索结果页

为什么要做动静分离?

- **访问路径差异：**静态页面访问路径短，可在几毫秒内完成；动态页面需经过业务逻辑计算、数据库查询等，通常需要几十甚至几百毫秒。
- **资源消耗不同：**动态请求消耗应用服务器CPU和数据库连接资源；静态请求主要是I/O和带宽资源。
- **缓存友好性：**静态资源可被浏览器、CDN等多级缓存；动态内容难以缓存

CDN加速-就近访问

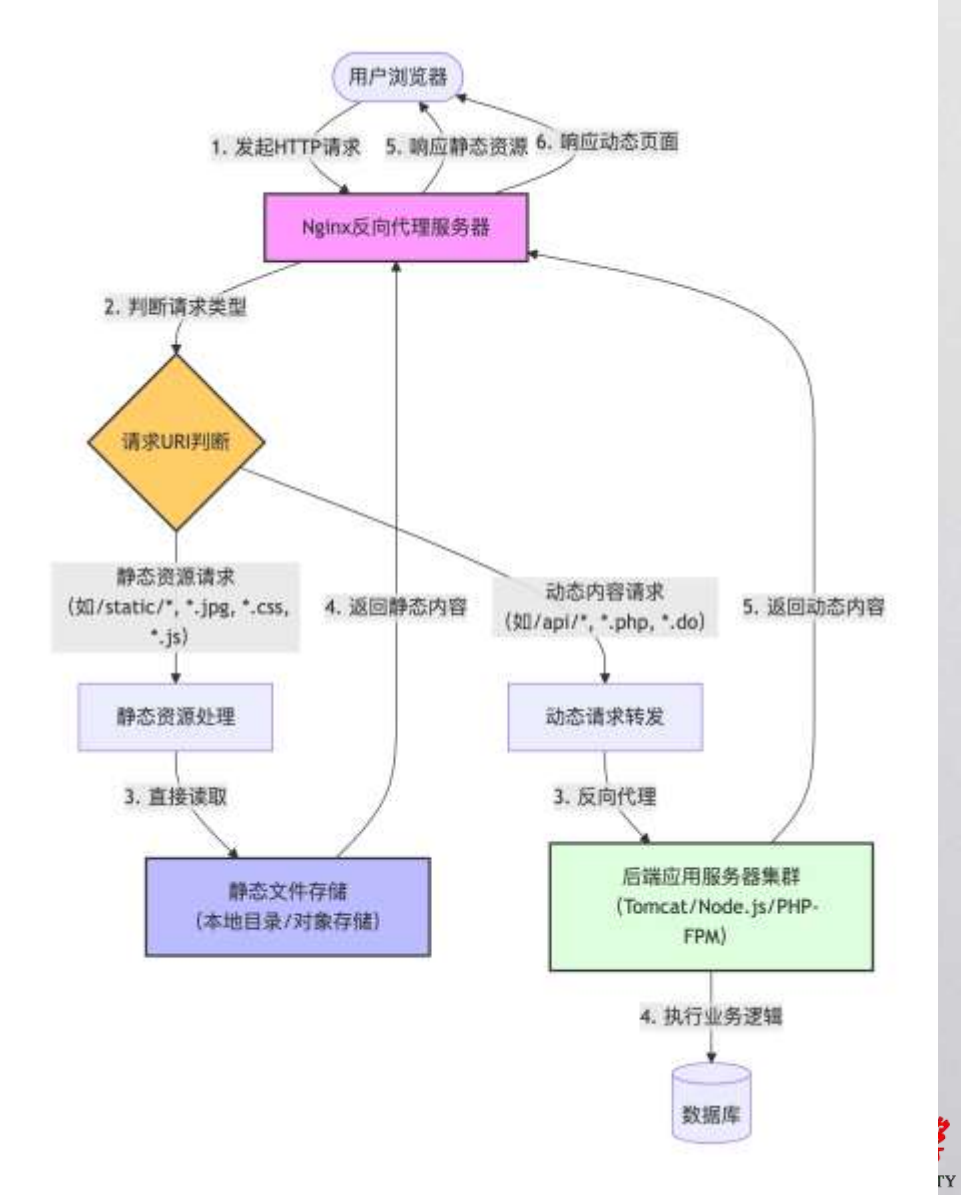


- **传统**CDN
  - 只能加速静态资源
  - 静态资源缓存在边缘节点
- **全站加速CDN**
  - 可同时加速静态和动态内容
  - 动态内容并不缓存内容，而是优化路径完成

使用Nginx进行动静分离

核心优势

- **降低应用服务器负载**：静态请求由 Nginx 直接处理，不占用后端应用资源。
- **提升响应速度：**Nginx 处理静态文件效率极高（基于 epoll 异步非阻塞模型）。
- **缓存策略灵活：**可针对静态资源设置强缓存（expires），减少重复请求。



使用Nginx进行动静分离

- 应用的前后端分开部署
- 前端的代码和文件直接通过Nginx访问
- 后端资源访问转发到应用服务器的端口上

```
server {
    listen 80;
    server_name example.com;
    # 静态资源处理
    location /static/ {
        alias /var/www/static/;          # 静态文件目录
        expires 30d;                     # 浏览器缓存30天
        access_log off;                  # 静态资源不记录访问日志
    }
    location ~* \.(jpg|jpeg|png|gif|css|js|ico)$ {
        root /var/www/static;            # 也可用正则匹配常见后缀
        expires 30d;
        access_log off;
    }

    # 动态请求转发
    location / {
        proxy_pass http://backend_servers;          # 应用服务器集群
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

4、 负载均衡

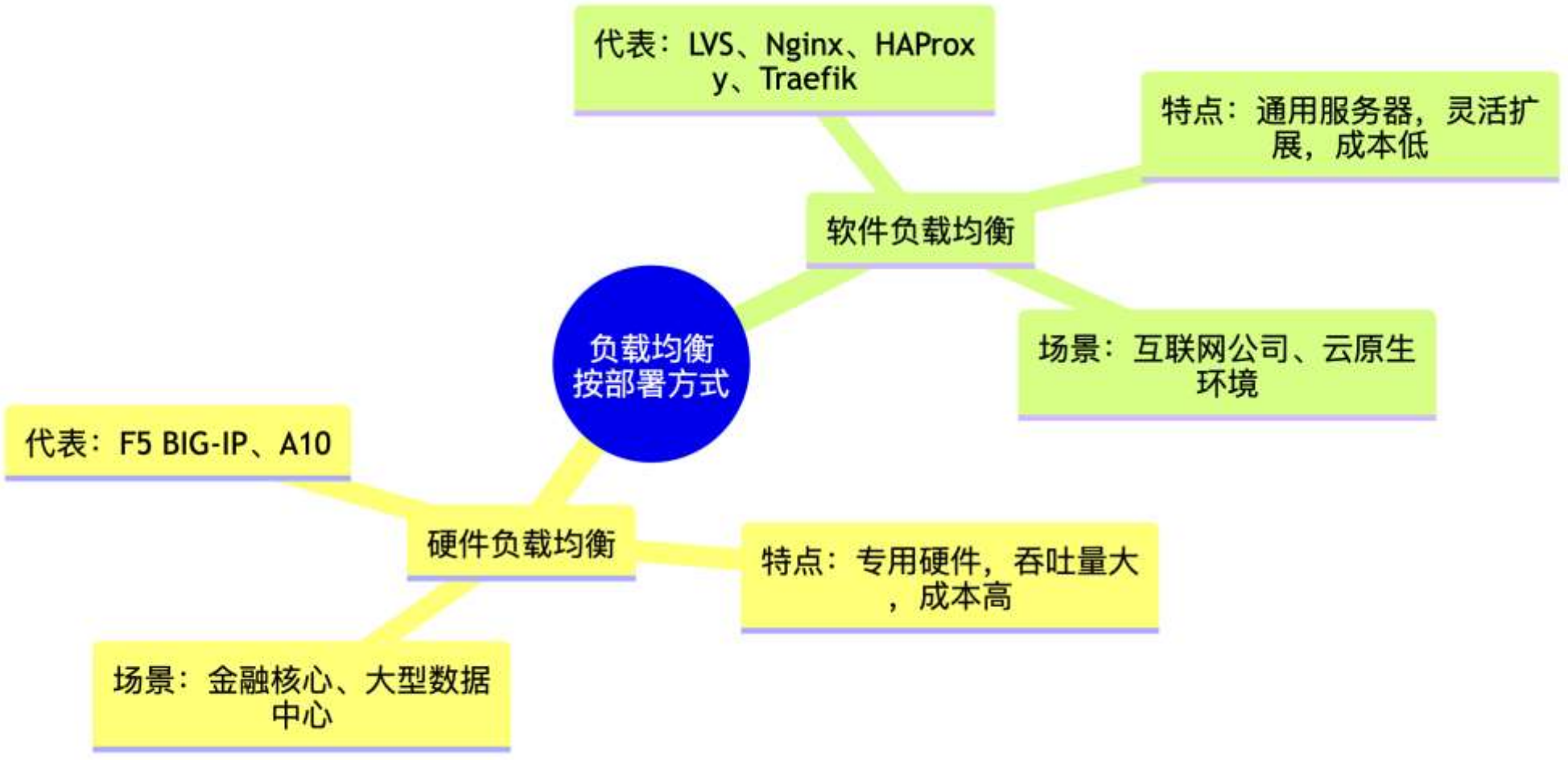
负载均衡

什么是负载均衡?

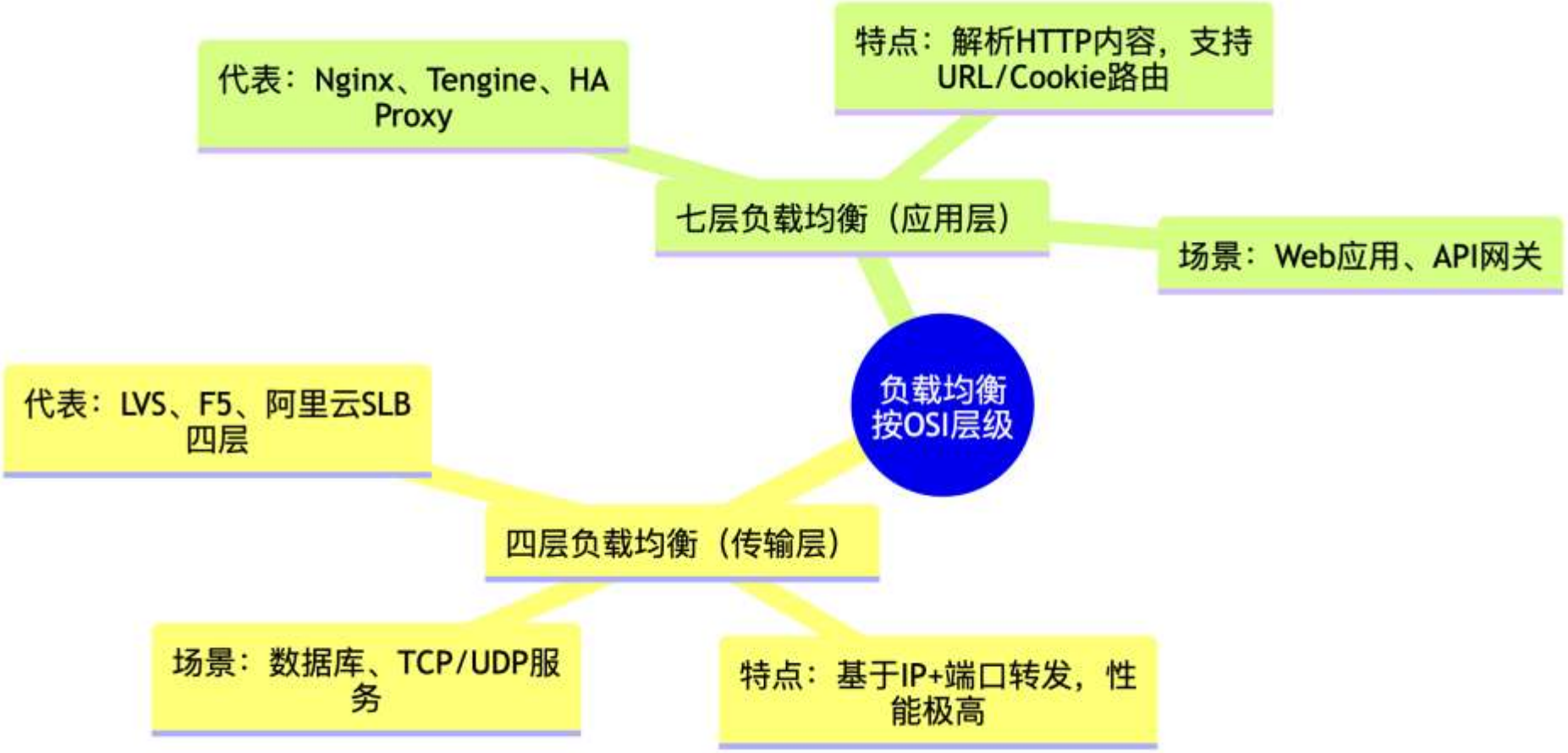
- 负载均衡是由多台服务器以对称的方式组成一个服务器集合，每台服务器都具有等价的地位，都可以单独对外提供服务。
- 通过某种负载分担技术，将外部发送来的请求均匀分配到对称结构中的某一台服务器上，而接收到请求的服务器独立地回应客户的请求。
- 负载均衡 = 统一入口 + 二次分发

需求	问题	负载均衡的解决方案
高性能	单台服务器处理能力有限	多台服务器组成集群，总处理能力线性扩展
高可用	单点故障导致服务中断	多台服务器互为备份，故障时自动切换
高扩展	业务增长需要扩容	随时添加服务器到集群，对外透明

负载均衡的分类


负载均衡
按部署方式
软件负载均衡
代表：LVS、Nginx、HAProxy、Traefik
特点：通用服务器，灵活扩展，成本低
场景：互联网公司、云原生环境
硬件负载均衡
代表：F5 BIG-IP、A10
特点：专用硬件，吞吐量大，成本高
场景：金融核心、大型数据中心

负载均衡的分类


负载均衡
按OSI层级
七层负载均衡（应用层）
代表：Nginx、Tengine、HA Proxy
特点：解析HTTP内容，支持URL/Cookie路由
场景：Web应用、API网关
四层负载均衡（传输层）
代表：LVS、F5、阿里云SLB四层
场景：数据库、TCP/UDP服务
特点：基于IP+端口转发，性能极高

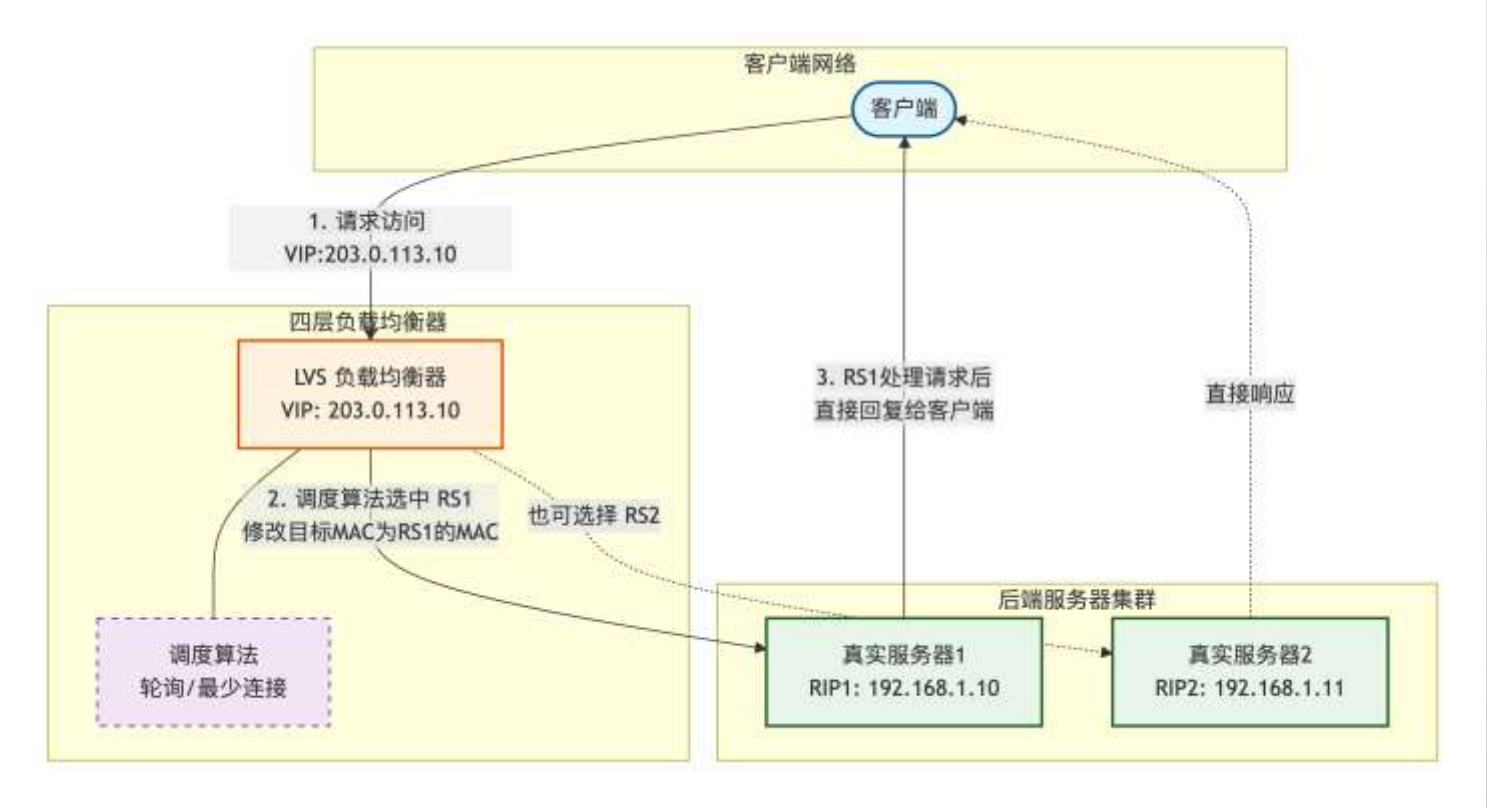
四层负载均衡

➢ 工作原理

- 基于IP地址和端口号进行转发，仅修改报文中的目标IP和源IP地址，不解析报文内容

➢ 典型实现

- LVS（Linux Virtual Server）
- Keepalived
- F5（硬件实现）



四层负载均衡

核心优势

- 性能极高，吞吐量大
- 只需处理TCP/UDP协议，开销小
- 三次握手由客户端和后端服务器直接建立

适用场景

- 高性能要求、简单协议转发、数据库负载均衡

七层负载均衡

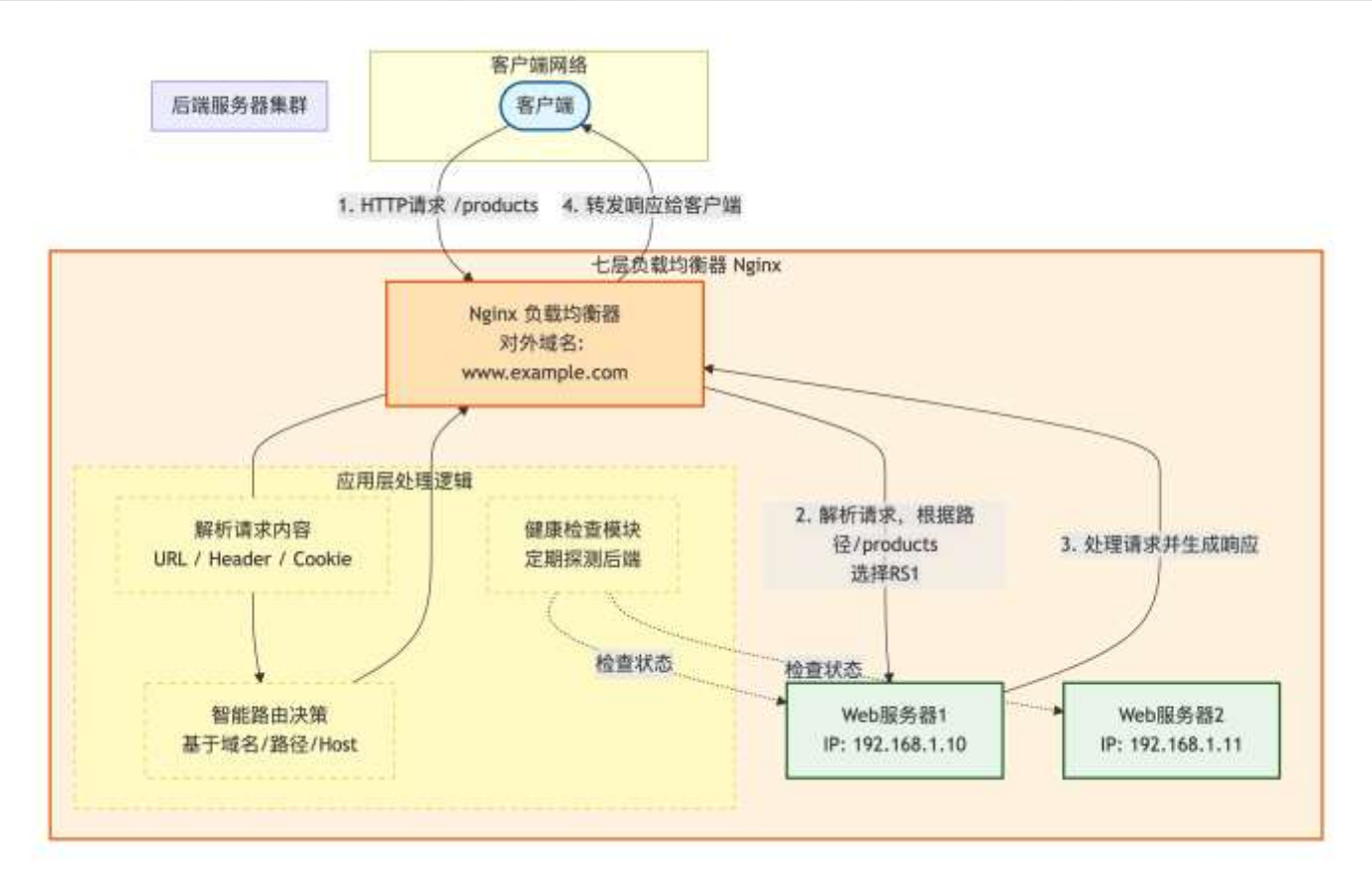
➢ 工作原理

- 采用**反向代理**模式。能够解析应用层协议（HTTP/HTTPS），根据报文内容（URL、Cookie、Header）转发

➢ 典型实现

- Nginx
- HAProxy
- Tengine（淘宝基于Nginx优化）

反向代理：作为服务器的代理人
正向代理：作为客户的代理人



七层负载均衡

核心优势

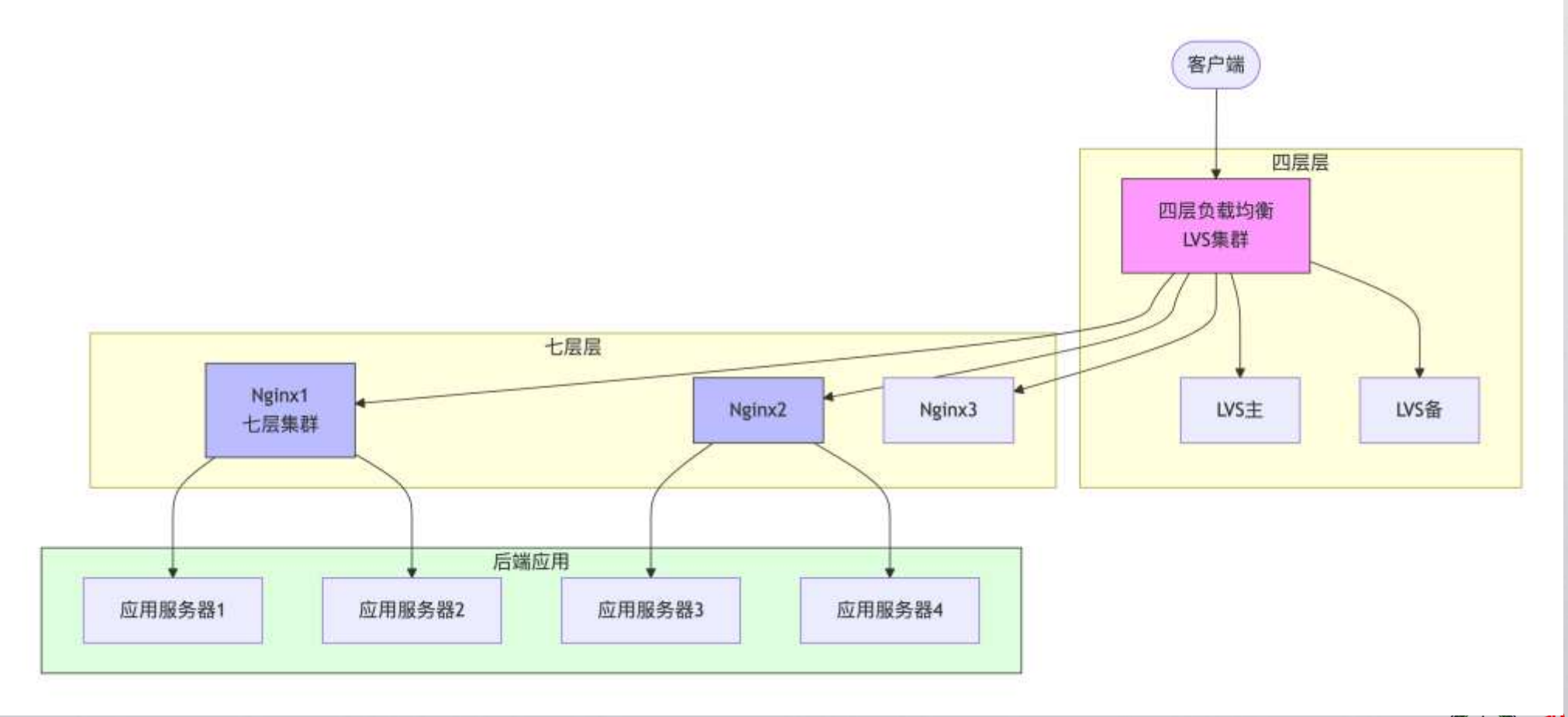
- 内容感知，可实现智能路由
- 支持SSL卸载（加解密）
- 可实现会话保持、缓存、重写等高级功能

适用场景

- Web 应用负载均衡（如 Nginx + Tomcat）
- API 网关（路由、认证、限流）
- 微服务架构的流量入口
- SSL 卸载：将 HTTPS 加解密集中在负载均衡层，减轻后端服务器负担

负载均衡组合

- 七层负载均衡 + 四层负载均衡：四层在前抗流量洪峰，七层在后做精细化分发。



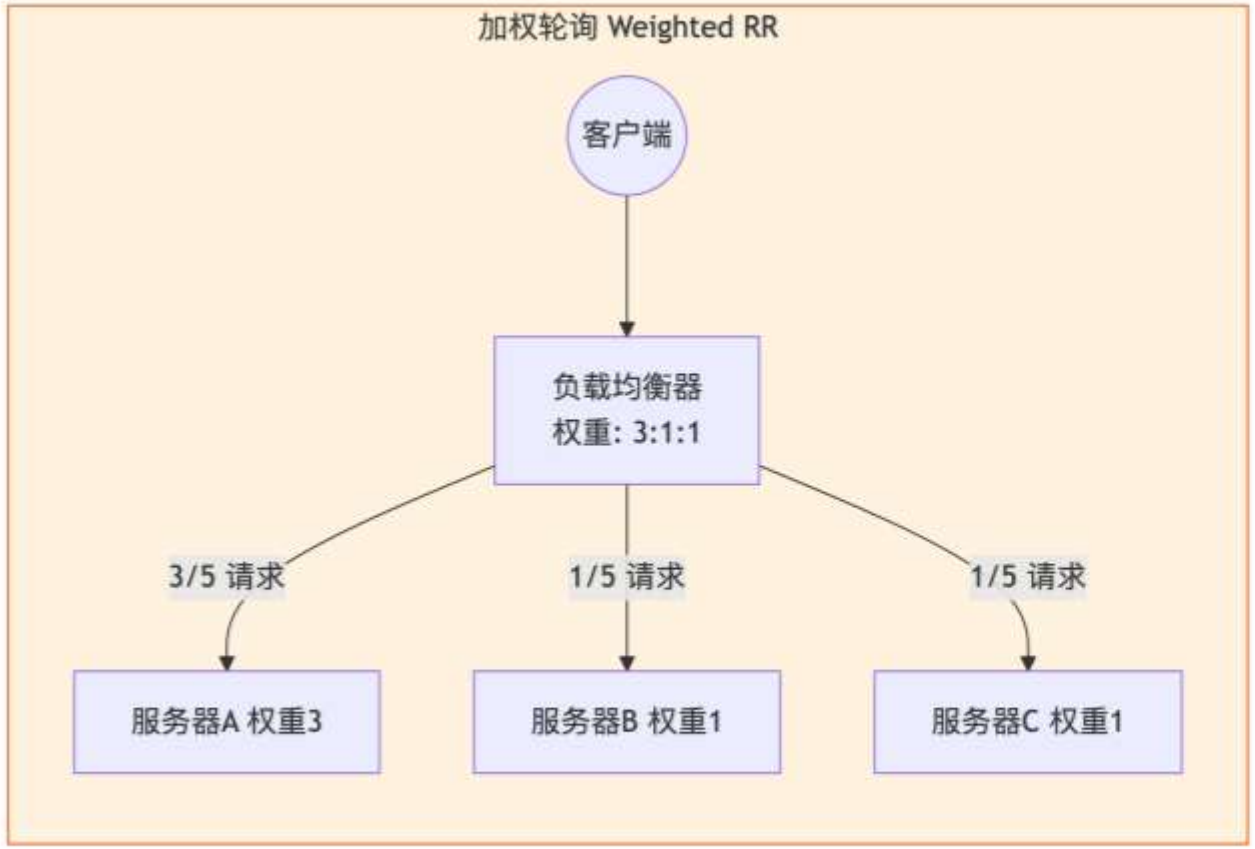
武漢大學
WUHAN UNIVERSITY

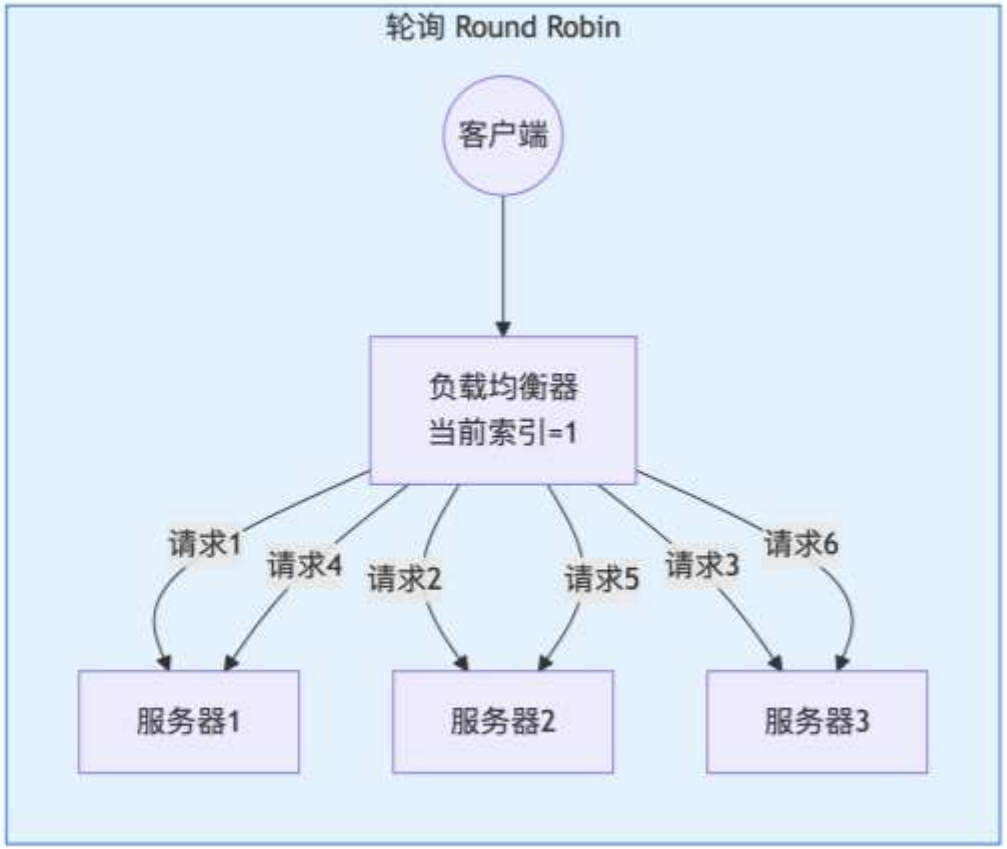
负载均衡算法

- 负载均衡器需要决定将下一个请求转发给哪台后端服务器。不同的算法直接影响系统的吞吐量、响应时间、资源利用率和稳定性。

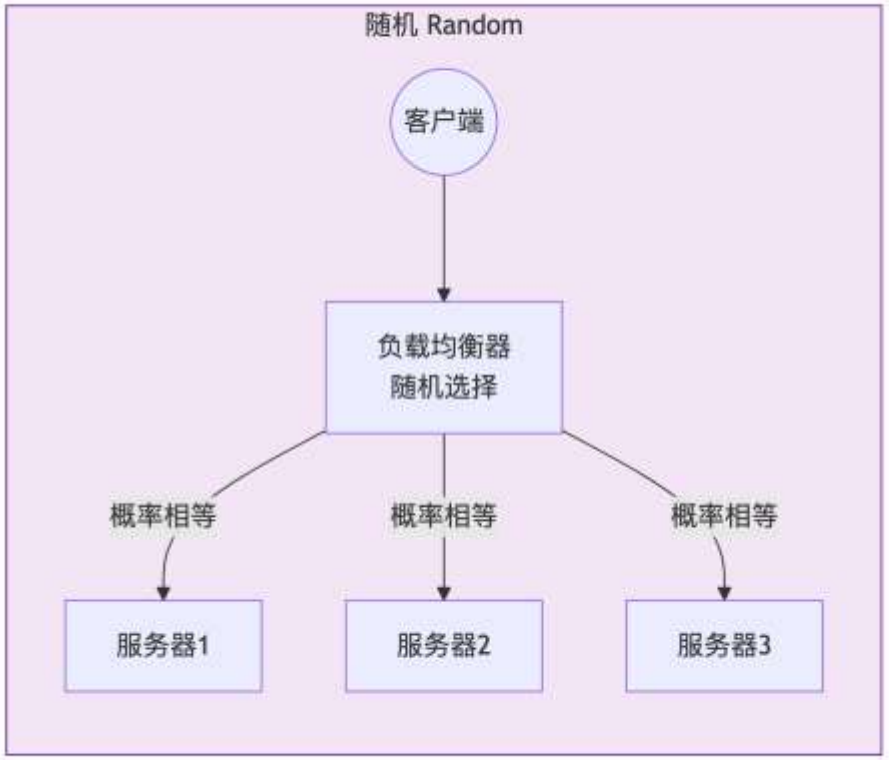
类型	定义	特点	典型算法
静态算法	不考虑服务器实时状态，仅按固定规则分配	实现简单，开销小；无法应对负载变化	轮询、加权轮询、IP哈希、随机
动态算法	根据服务器实时负载指标（连接数、响应时间等）动态调整	负载分配更均衡；需要实时采集数据，开销稍大	最少连接、加权最少连接、响应时间

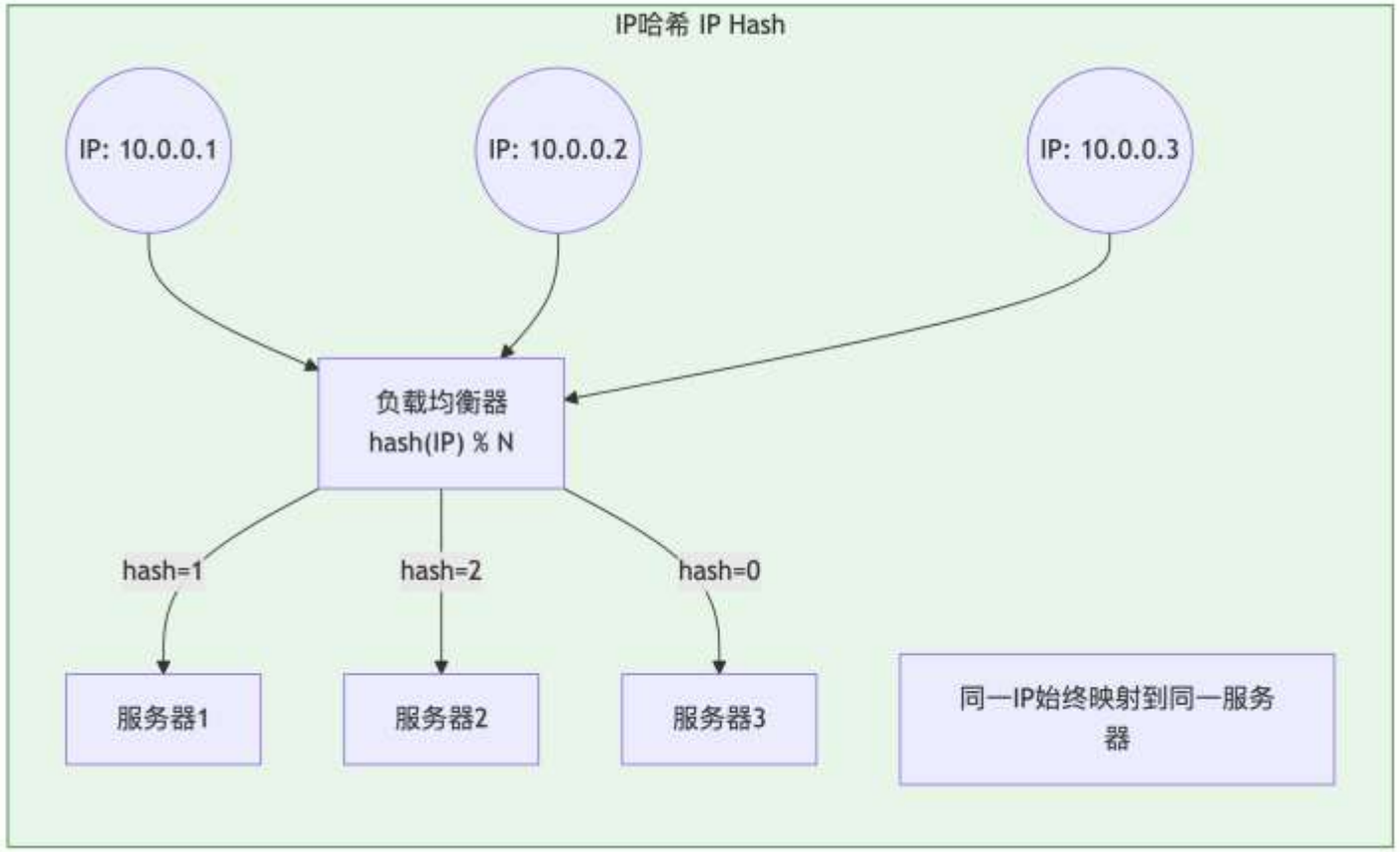
静态负载均衡算法


加权轮询 Weighted RR
客户端
负载均衡器
权重: 3:1:1
3/5 请求
1/5 请求
1/5 请求
服务器A 权重3
服务器B 权重1
服务器C 权重1


轮询 Round Robin
客户端
负载均衡器
当前索引=1
请求1
请求4
请求2
请求5
请求3
请求6
服务器1
服务器2
服务器3

静态负载均衡算法

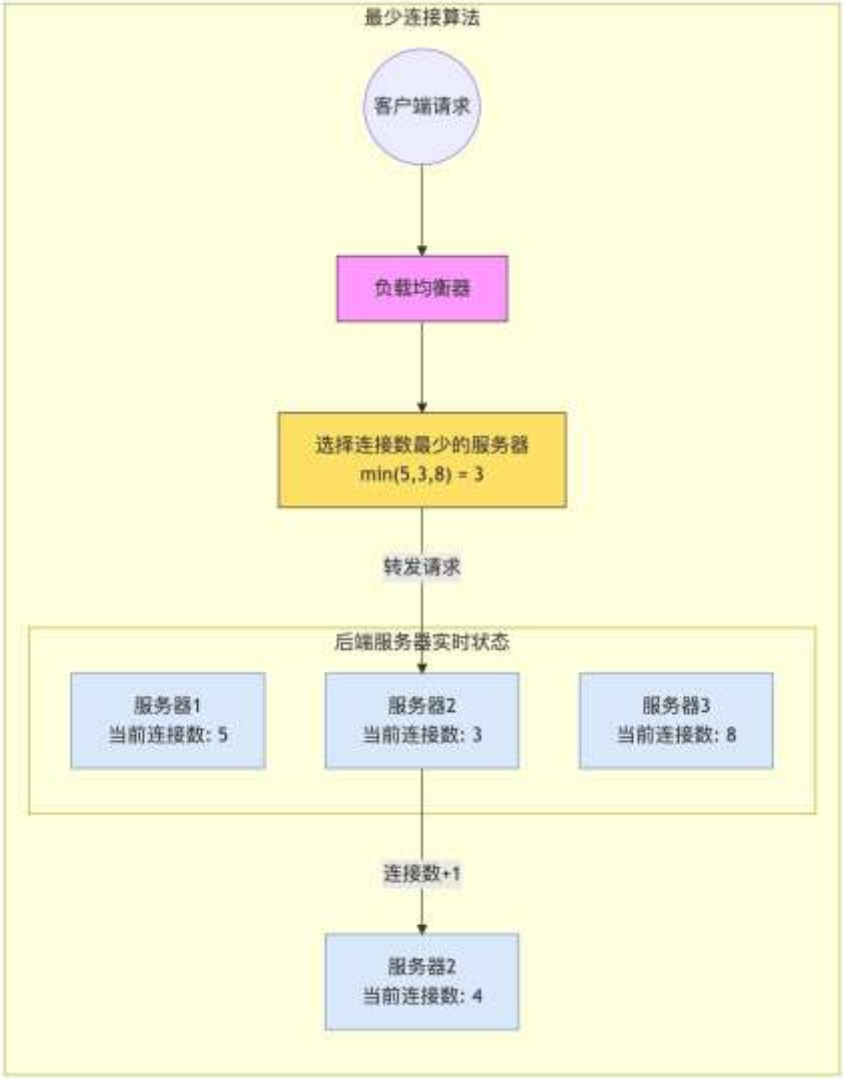

随机 Random
客户端
负载均衡器
随机选择
概率相等
概率相等
概率相等
服务器1
服务器2
服务器3

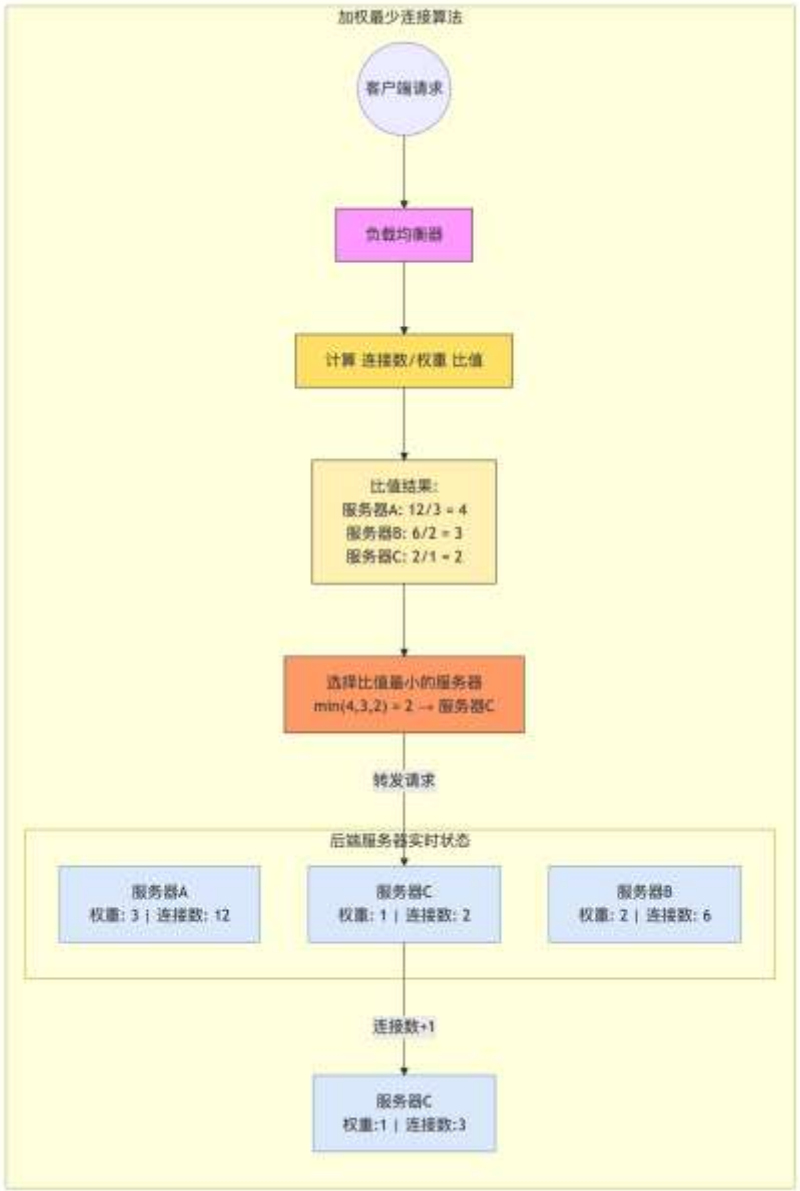

IP哈希 IP Hash
IP: 10.0.0.1
IP: 10.0.0.2
IP: 10.0.0.3
负载均衡器
hash(IP) % N
hash=1
hash=2
hash=0
服务器1
服务器2
服务器3
同一IP始终映射到同一服务器

静态负载均衡算法

算法	原理	特点
轮询	请求依次循环分配给每台服务器	完全平均，实现简单；不考虑服务器当前负载
加权轮询	根据预设权重分配请求，权重高的服务器获得更多请求	适应服务器性能异构的场景，权重需人工配置
IP哈希	对客户端IP进行哈希计算，根据哈希值映射到固定服务器	同一客户端IP始终访问同一服务器（会话保持）
随机	从服务器列表中随机选择一个	大流量下趋于平均，实现极简

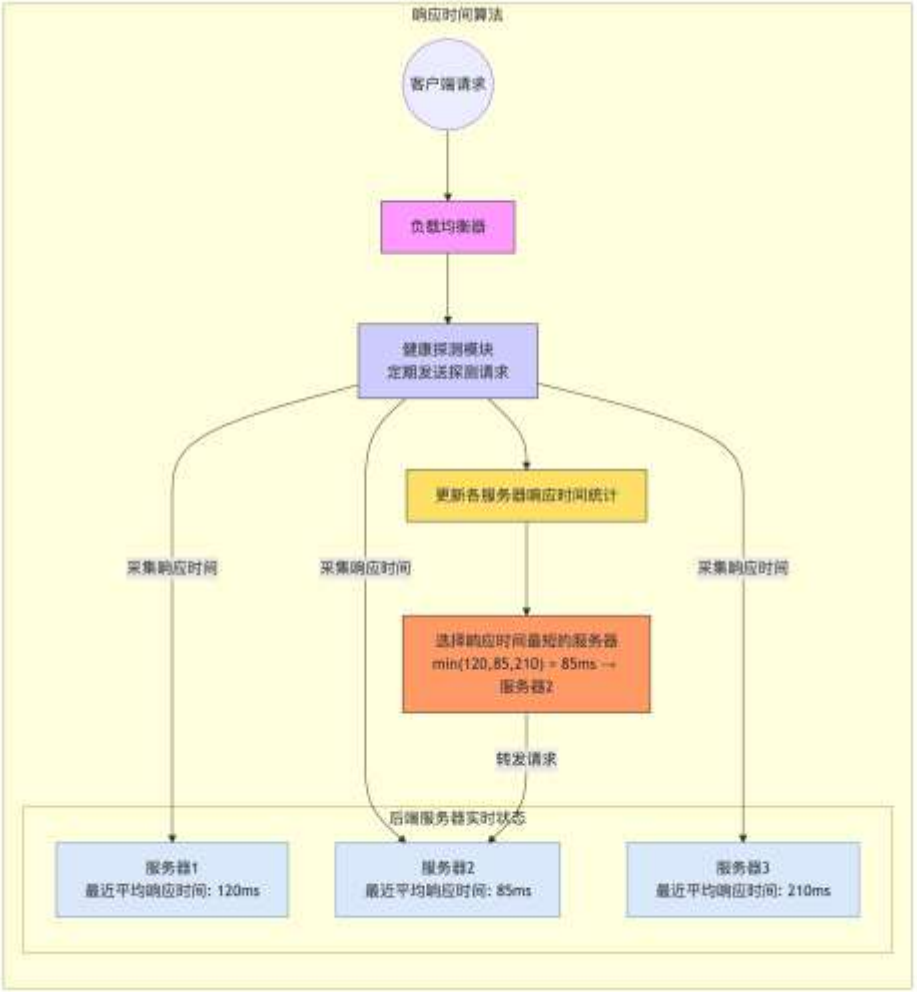
动态负载均衡算法



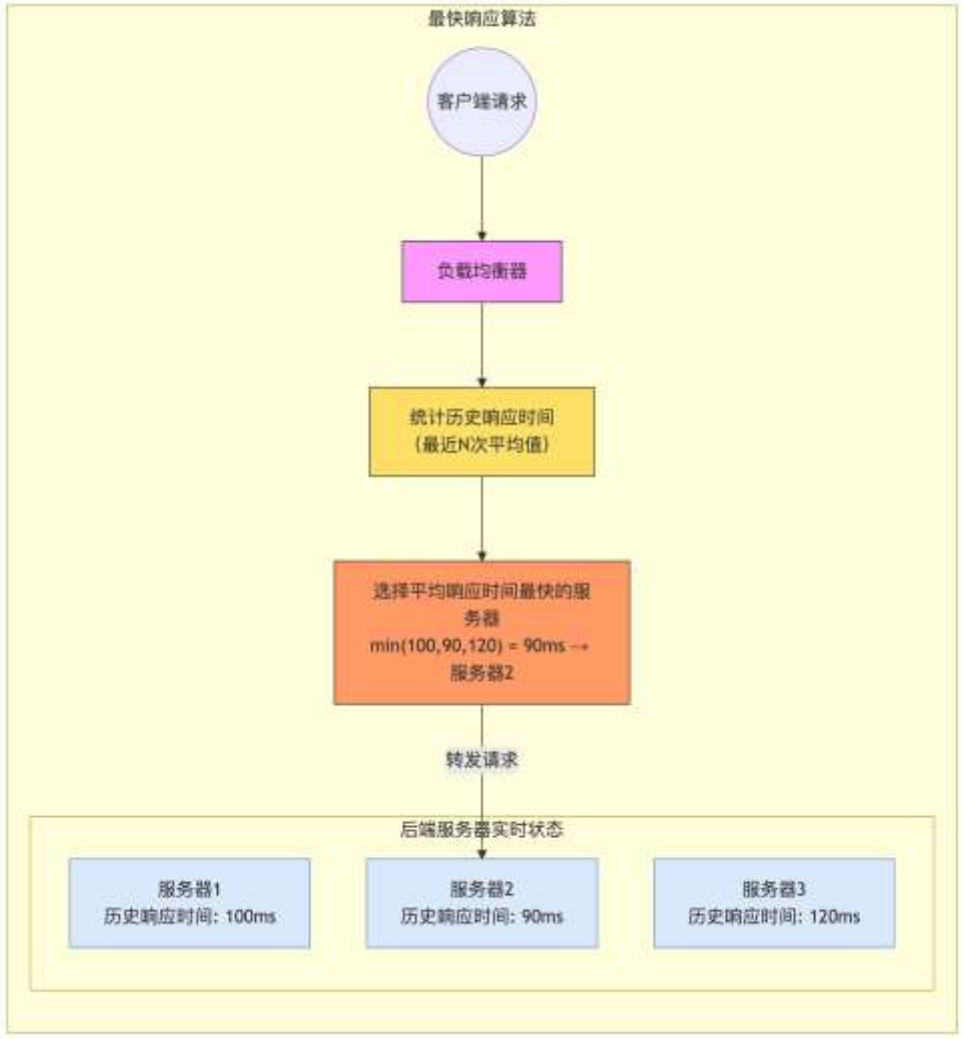


武漢大學
WUHAN UNIVERSITY

动态负载均衡算法



响应时间算法：基于实时探测



最快响应算法基于历史统计

武漢大學
WUHAN UNIVERSITY

动态负载均衡算法

算法	核心原理	适用场景
最少连接	将新请求分配给当前活跃连接数最少的服务器	长连接服务（FTP、WebSocket、数据库连接池）
加权最少连接	计算每台服务器的 连接数/权重，选择比值最小的服务器	服务器性能不同且连接时长差异大的场景
响应时间	选择最近探测响应时间最短的服务器	延迟敏感的在线服务（API网关、搜索服务）
最快响应	基于历史响应时间统计，选择平均响应最快的服务器	与响应时间类似，但更适用于响应时间波动较大的环境
预测算法	根据历史负载趋势预测未来负载，选择预测负载最低的服务器	负载模式有明显周期性的业务（如定时促销、夜间离线任务）

不同场景的算法选择

业务场景	推荐算法	原因
静态网站（短连接、请求快）	轮询 / 加权轮询	请求处理时间相近，简单公平即可
API服务（响应时间敏感）	响应时间 / 最少连接	优先保证低延迟，避免慢服务器拖累整体
WebSocket / 即时通讯（长连接）	最少连接	避免连接集中在某几台服务器
需要会话保持（登录态）	IP哈希 / Cookie插入（七层）	确保用户请求始终由同一服务器处理
服务器性能异构集群	加权轮询 / 加权最少连接	充分发挥高性能服务器能力
下载服务 / 大文件传输	最少连接	避免某台服务器因大流量过载
数据库负载均衡	最少连接 / 响应时间	数据库连接长时间占用，需动态调整

负载均衡算法组合

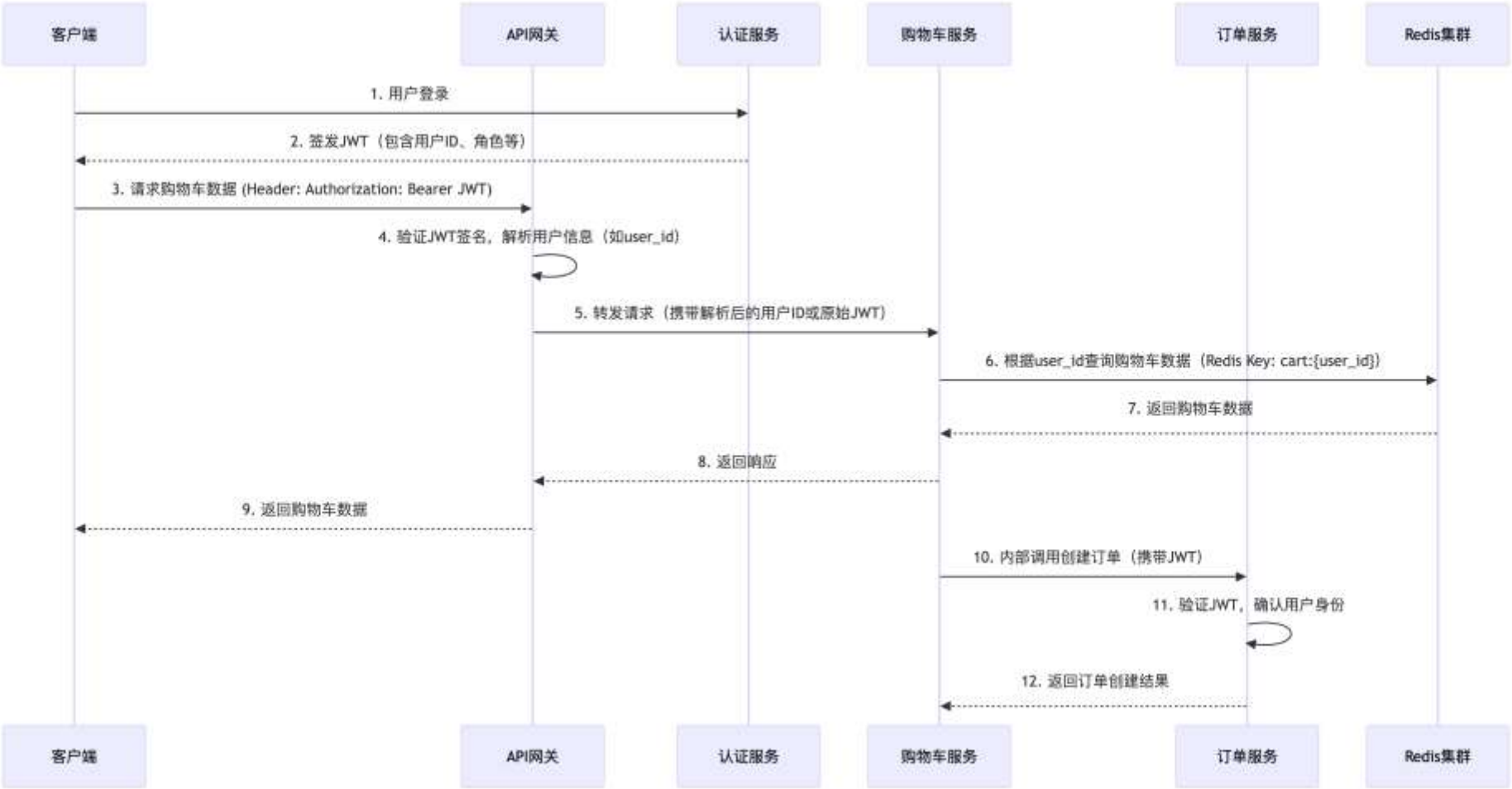
- 不同路径不同算法：如Nginx可配置不同location使用不同算法。
  - /api → 最少连接（保障响应速度）
  - /static → 轮询（简单快速）
  - /user → IP哈希（保持登录会话）

```
upstream api_servers {
    least_conn;              # API用最少连接算法
    server 192.168.1.10:8080;
    server 192.168.1.11:8080;
}
upstream static_servers {
    round_robin;             # 静态资源用轮询
    server 192.168.1.20:80;
    server 192.168.1.21:80;
}
upstream user_servers {
    ip_hash;                 # 用户中心用IP哈希保持会话
    server 192.168.1.30:8080;
    server 192.168.1.31:8080;
}
server {
    listen 80;
    server_name example.com;
    location /api/ {
        proxy_pass http://api_servers;
    }
    location /static/ {
        proxy_pass http://static_servers;
        expires 30d;         # 静态资源缓存
    }
    location /user/ {
        proxy_pass http://user_servers;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

负载均衡的会话保持

- 会话保持
  - 指将来自同一客户端的请求始终分发到同一台后端服务器，以保证会话状态（如登录信息、购物车数据）不丢失。
- **主流实现方式**
  - 源地址哈希：四层可用，但可能不均。
  - Cookie插入：七层负载均衡生成Cookie，客户端后续请求携带，保证同会话固定到同一服务器。
  - Redis集中存储会话：服务器无状态，会话存储在共享Redis中，任何服务器都可处理。
  - 无状态Token：客户端存储Token（如JWT），Token包含用户信息+签名，服务器验证签名即可，不存储会话（常用）

负载均衡的会话保持

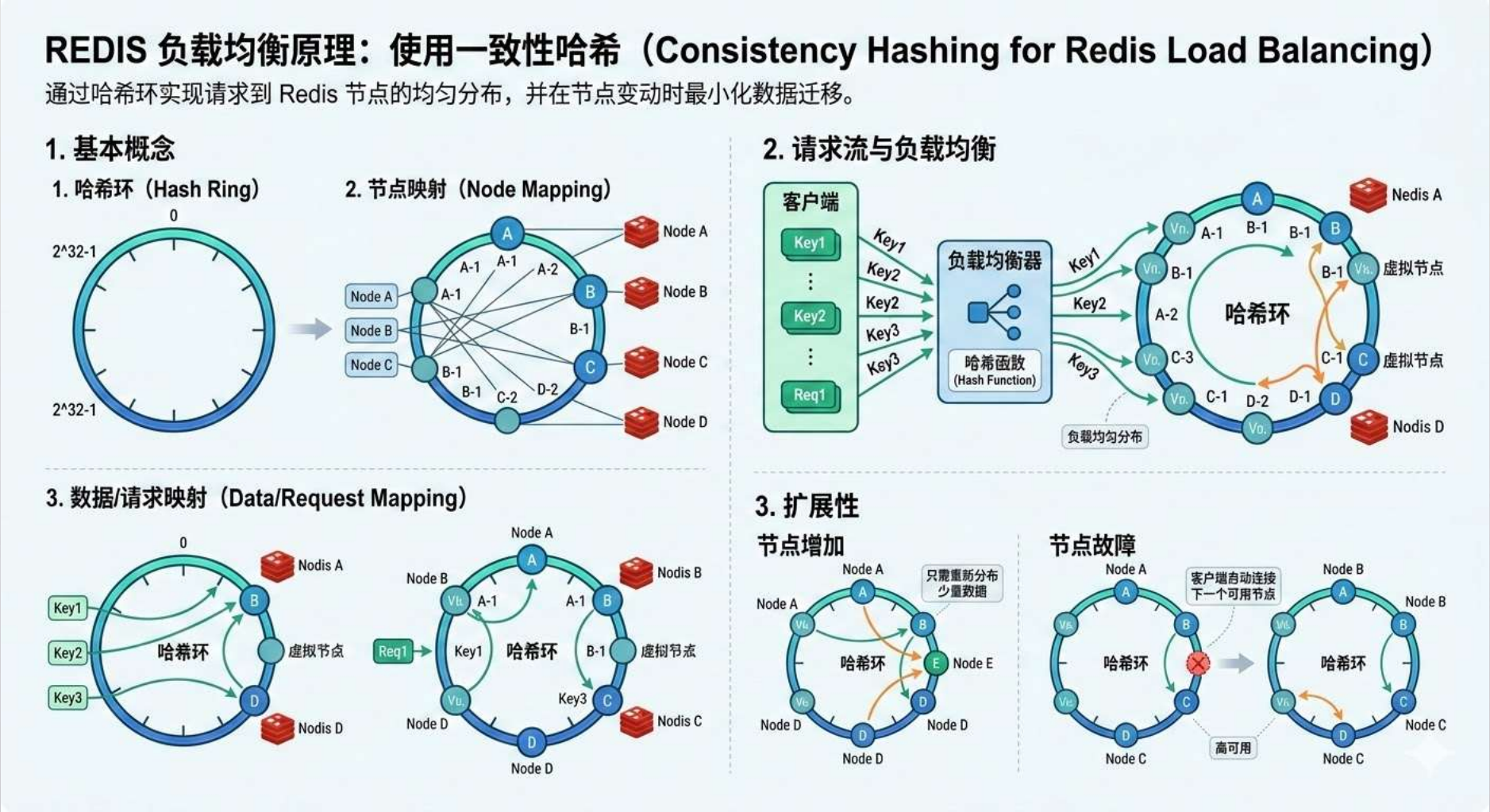


武漢大學
WUHAN UNIVERSITY

负载均衡的一致性哈希

- 一致性哈希：将请求的Key（如IP、URL）哈希到环上，服务器也映射到环上，请求落在最近的服务器上。

- 优点：支持节点动态扩缩容。当服务器增减时，只有少量请求需要重新分配。
- 适用场景：分布式缓存（如Redis分片）、CDN边缘节点。



5、 缓存

缓存

- 缓存的定义
  - 缓存（Cache）是将数据存储在临时存储介质（通常是内存）中，以便后续能够更快访问的技术。
- 缓存的价值
  - 降低延迟：将响应时间从几十毫秒降至毫秒甚至微秒级
  - 减轻数据库压力：拦截90%以上的读请求
  - 提升系统吞吐：用低成本的内存换取高并发能力

对比维度	直接访问数据库	访问缓存
存储介质	磁盘（机械/SSD）	内存（DRAM）
访问延迟	10-50ms（SSD）或更高	0.1-1ms
吞吐能力	几百~几千 QPS	数万~数十万 QPS
成本	相对低廉	较高

本地缓存 vs 分布式缓存

维度	本地缓存	分布式缓存
代表产品	Caffeine、Guava Cache、Ehcache	Redis、Memcached
存储位置	应用进程内内存	独立服务器内存
访问速度	极快（纳秒级，无网络开销）	快（毫秒级，有网络开销）
数据一致性	各节点独立，需自行同步	所有应用共享同一份数据
适用场景	静态配置、热点数据二级缓存	全局共享数据、会话存储

分布式缓存Redis

➢ Redis的性能表现

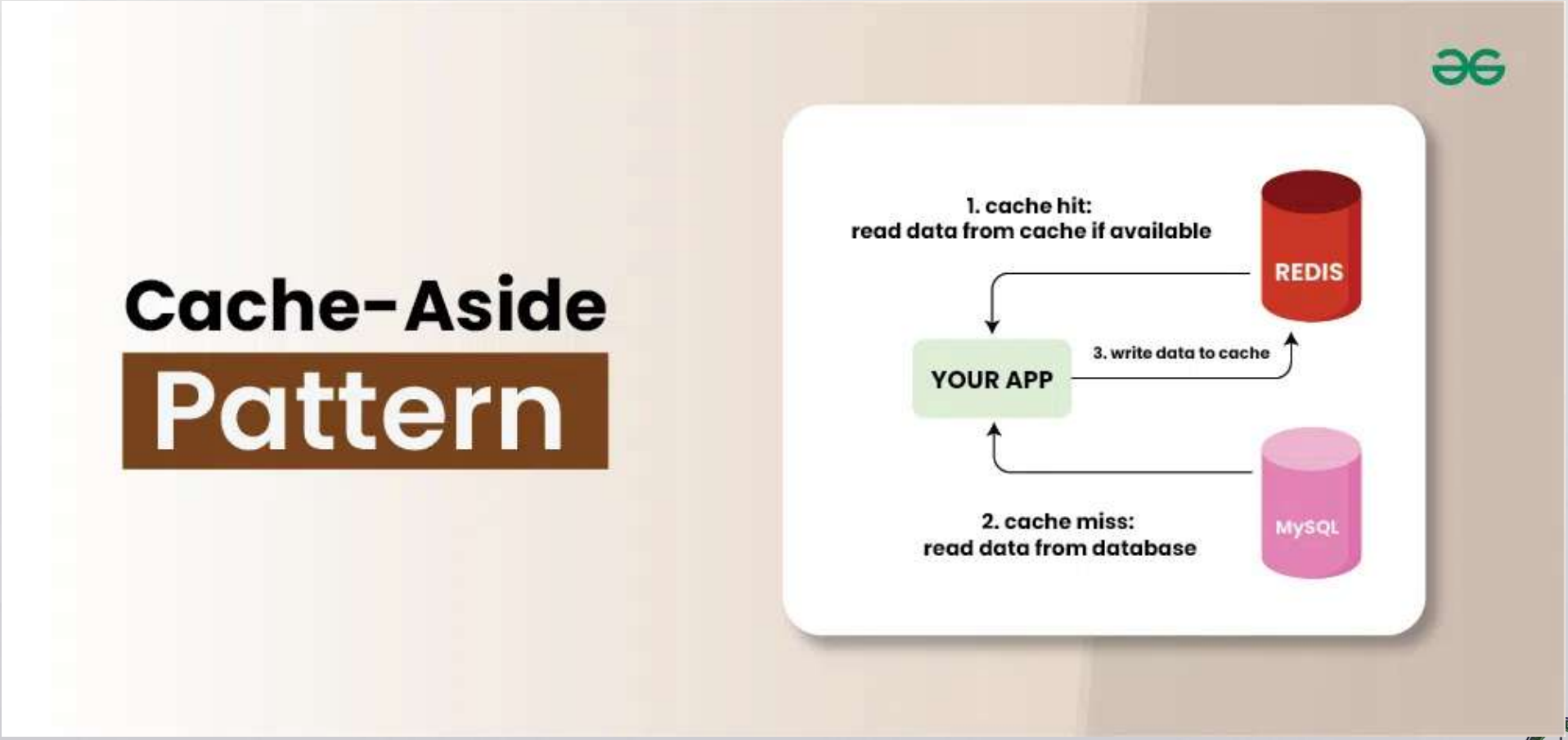
- 单节点读写QPS可达10万+，平均响应时间<1毫秒，阿里双11峰值调用量达5亿次/秒

技术特性	说明	对高并发的贡献
纯内存存储	数据全部在内存中操作	避免磁盘I/O瓶颈
单线程+多路复用	基于epoll/Reactor模型，单线程避免锁竞争	上下文切换开销小，高并发下稳定
高效数据结构	支持String、Hash、List、Set、ZSet等	针对不同场景优化，多数操作O(1)
原子操作	所有指令都是原子的	天然支持高并发计数、限流等场景

Redis教程：https://www.runoob.com/redis/redis-tutorial.html

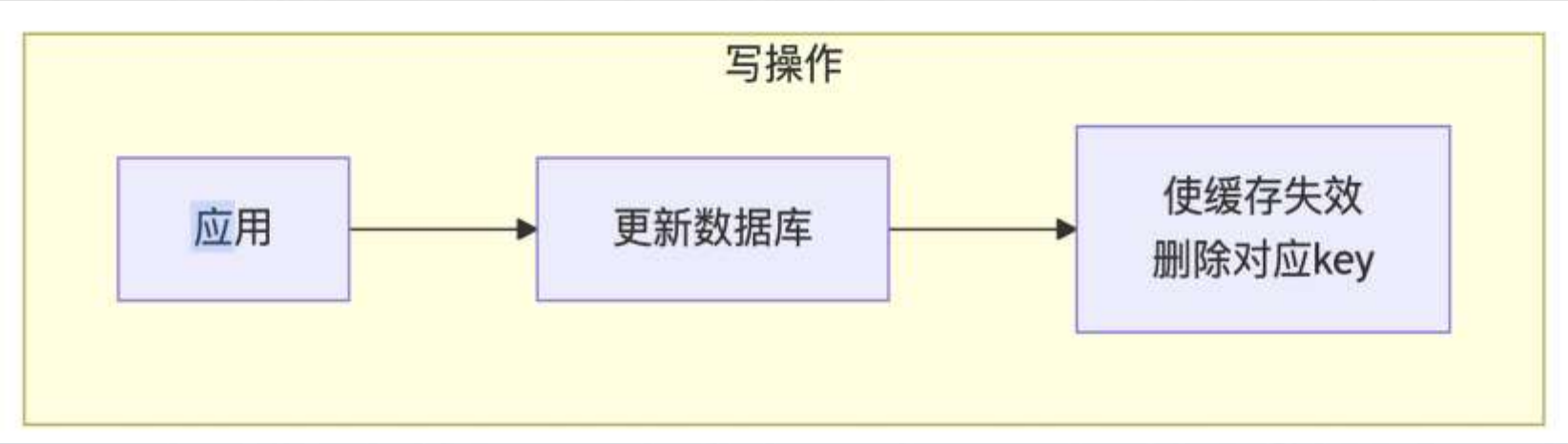
缓存读写模式

（1）Cache Aside——旁路缓存模式（最常用）



武漢大學
WUHAN UNIVERSITY

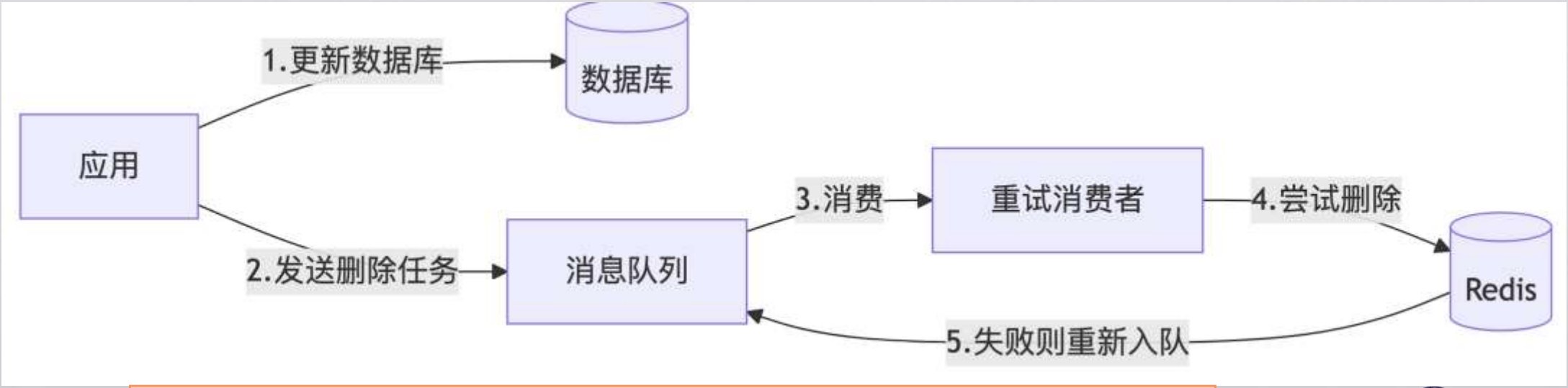
Cache Aside——旁路缓存模式



- 为什么先更新数据库，再删除缓存？
  - 错误做法：先删缓存，再更新数据库 → 并发查询可能把旧数据读回缓存
  - 正确做法：先更新数据库，再删缓存 → 短暂不一致后，下次查询会加载新数据
  - Facebook论文验证：Memcache在Facebook的实践中采用此策略

Cache Aside——旁路缓存模式

- 删除缓存失败怎么办?
  - 删除缓存失败（例如网络抖动、Redis 超时或宕机）
  - 策略1：重试机制 + 过期时间兜底
  - 策略2：通过监听（可以用Canal）数据库的变更日志（binlog），由独立服务后台删除缓存

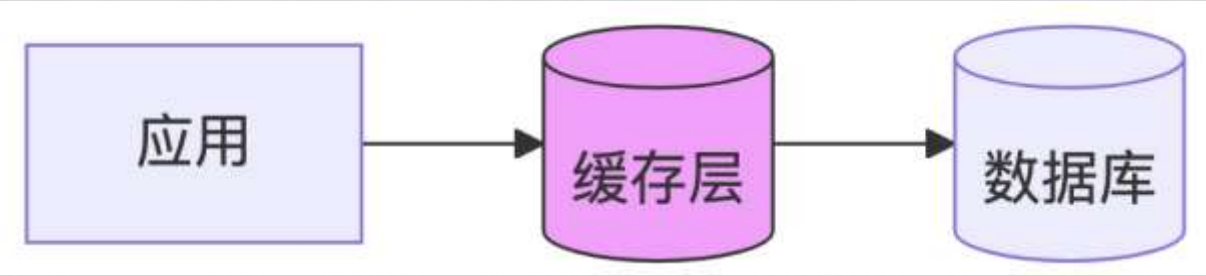


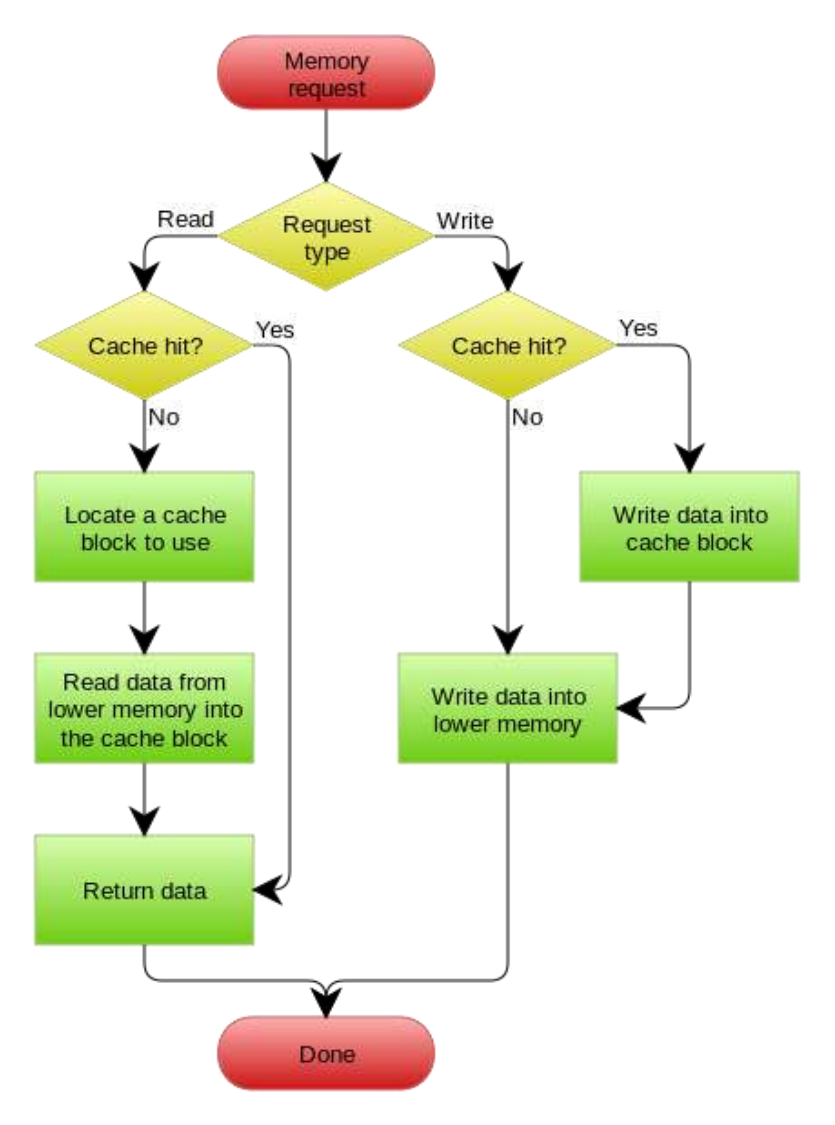
jedis.setex(key, value, 3600); // 设置1小时过期（Jedis库）

武漢大學 WUHAN UNIVERSITY

Read/Write Through——缓存代理模式

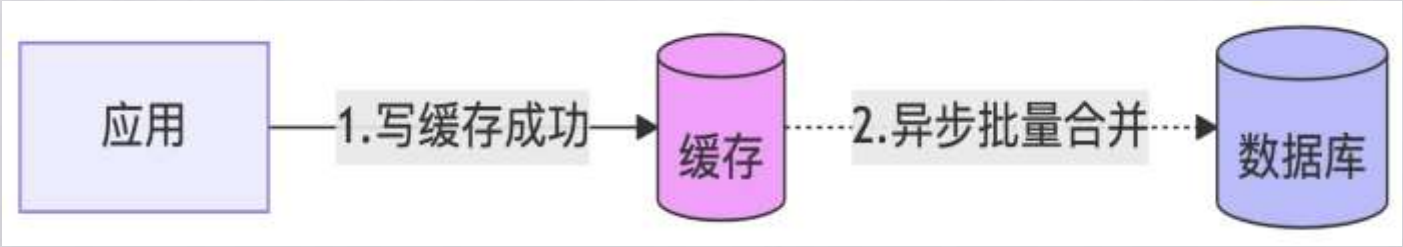
- 缓存代理模式
  - 读： 读未命中时，缓存自行加载数据库数据。应用无需关心缓存加载逻辑
  - 写： 写操作时，先更新缓存，缓存同步更新数据库。保证缓存与数据库强一致（同步写）





Write Behind——异步写回模式

- 异步写回
  - 先更新缓存，异步批量写入数据库
- 优点
  - 性能极高：写操作只操作内存，响应极快
  - 合并写操作：多次更新可合并为一次数据库写
  - 削峰填谷：吸收写入峰值



- 缺点
  - 数据可能丢失：缓存宕机时未落盘数据丢失
  - 最终一致性：数据库数据有延迟
- 适用场景
  - 点赞计数、访问统计
  - 日志收集
  - 购物车合并保存入数据库

武漢大學 WUHAN UNIVERSITY

三种缓存模式对比

模式	读流程	写流程	一致性	性能	复杂度	场景示例
Cache Aside	缓存未命中时应用加载	先更新DB，再删缓存	强最终一致	高	低	电商商品详情页 用户资料缓存
Read/Write Through	缓存代理加载数据	缓存代理同步写数据	强一致（同步）	中	中	配置管理中心 游戏玩家状态
Write Behind	同Read Through	异步批量写DB	最终一致	极高	高	点赞/访问计数器 购物车合并保存 物联网设备数据上

高并发下的缓存问题

用户
缓存命中
缓存命中
缓存
(Redis)
缓存未命中
缓存未命中
数据库
(MySQL)
场景 1：缓存穿透
Cache
恶意查询
Key：ID -1
查询未命中
(不存在)
恶意查询
Key：ID -1
(该 Key 不存在)
数据库过载
(数据未找到)
查询不存在的数据，绕过缓存
并重复击中数据库。
原因：恶意攻击，代码逻辑错误。
解决方案：
布隆过滤器
"NULL"
缓存“NULL”
(短 TTL)
场景 2：缓存击穿
Cache
Cache
0
缓存到期
热点 Key：
秒杀商品
大量并发请求
单个高并发热点 Key 到期，
导致即时并发请求涌入数据库。
数据库崩溃
(单个 Key 未命中)
原因：热点 Key 在高峰负载期间到期。
解决方案：
LOCK
DB
分布式锁
(DB 查询)
逻辑永不过期
场景 3：缓存雪崩
Key
A
B
C
D
E
TTL：1 小时
TTL：1 小时
TTL：1 小时
TTL：1 小时
TTL：1 小时
时钟到 1 小时
全部到期
(TTL 随机化失效)
海量 Key 同时到期，
压垮数据库。
整个系统瘫痪
(海量请求未命中)
原因：大量 Key 并发到期，或 Redis 宕机。
解决方案：
随机化 TTL
(±5 分钟)
DB
熔断 / 限流

缓存穿透

- 缓存穿透
  - 请求查询不存在的数据，缓存和数据库都没有，每次请求都穿透到数据库
- 典型场景：
  - 恶意攻击：请求id=-1或超大id
  - 爬虫扫描不存在的资源
  - 业务数据尚未生成
- 危害
  - 数据库压力激增，可能导致宕机
  - 计算公式：QPS 10,000 × 穿透率90% = 数据库9,000次/秒

缓存穿透的解决方案

- 三种场景方案对比

方案	原理	优点	缺点
缓存空值	查询不到的数据也缓存，设短过期	实现简单，效果直接	可能缓存大量空key，占用内存
布隆过滤器	预加载所有合法key，请求前过滤	内存占用极小，拦截彻底	实现复杂，有误判率
参数校验	在入口层校验id合法性	简单快速	只能拦截明显非法请求

防御缓存穿透：缓存“NULL”

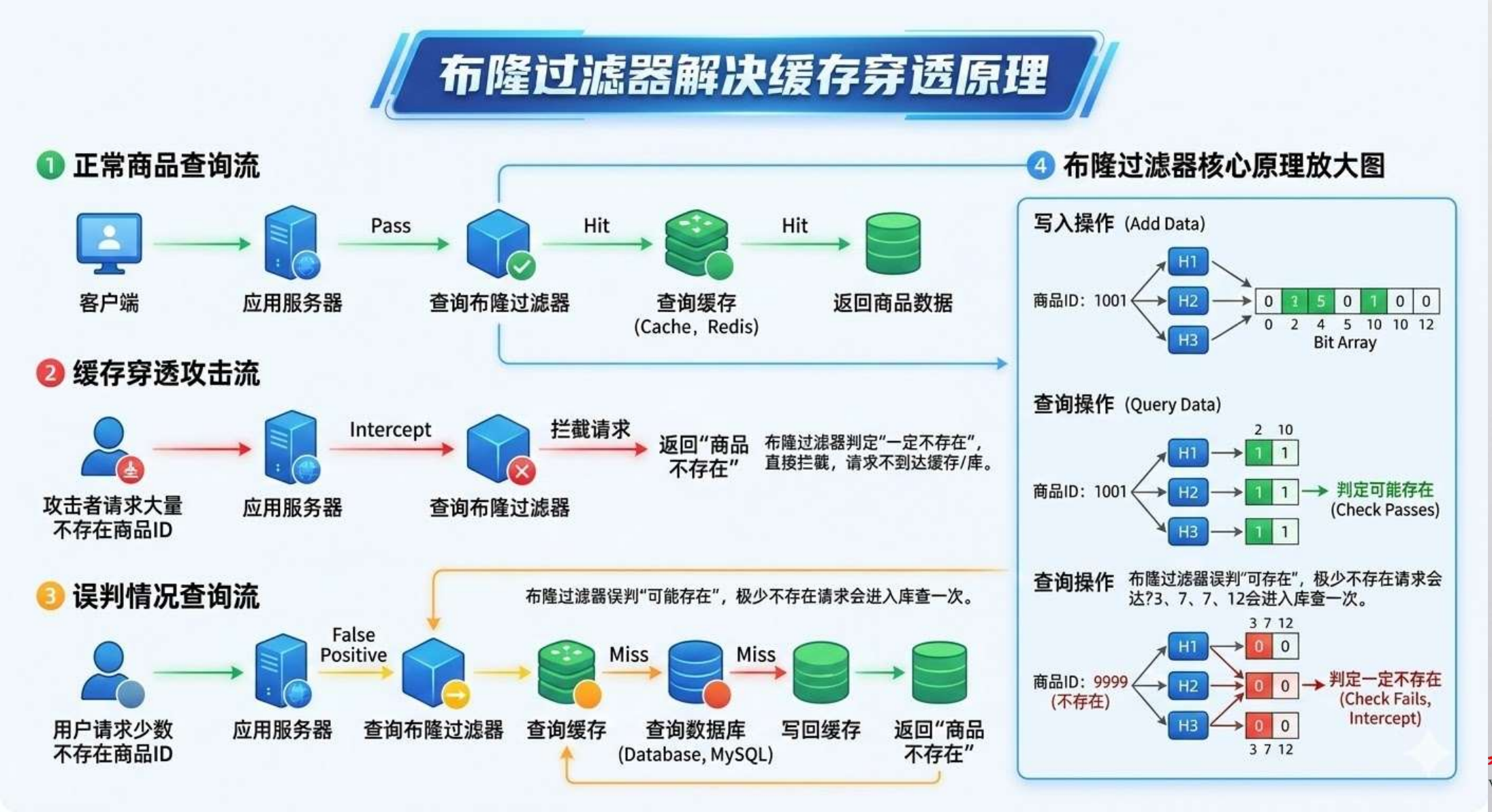
```java
public String getProduct(String productId) {
    String cacheKey = "product:" + productId;
    // 1. 从Redis缓存查询
    String cachedValue = redisTemplate.opsForValue().get(cacheKey);

    // 2. 如果缓存命中
    if (cachedValue != null) {
        // 如果缓存的是空值标记，表示数据库中不存在该商
        if ("NULL".equals(cachedValue)) {
            return null; // 或抛异常，或返回空对象
        }
        // 否则返回真实数据
        return cachedValue;
    }
```

```java
// 3. 缓存未命中，查询数据库
String dbValue = queryDatabase(productId);

// 4. 根据数据库结果处理缓存
if (dbValue == null) {
    // 数据库不存在，缓存空值，设置较短的过期时间（如5分钟）
    redisTemplate.opsForValue().set(cacheKey, "NULL", 5, TimeUnit.MINUTES);
    return null;
} else {
    // 数据库存在，缓存真实数据，设置正常的过期时间（如1小时）
    redisTemplate.opsForValue().set(cacheKey, dbValue, 1, TimeUnit.HOURS);
    return dbValue;
}
```

防御缓存穿透：布隆过滤器



防御缓存穿透：布隆过滤器

```
<dependency>
    <groupId>com.google.guava</group
    <artifactId>guava</artifactId>
    <version>32.1.3-jre</version>
</dependency>
```

```java
/**
 * 新增商品时，需要更新布隆过滤器
 */
public void addProduct(String productId, String nam
    // 先插入数据库
    // database.insert(...)
    // 再更新布隆过滤器
    bloomFilter.put(productId);
    // 可选：清除缓存（使下次查询重新加载）
    redisTemplate.delete("product:" + productId);
}
```

```java
/**
 * 查询商品信息（使用布隆过滤器防御穿透）
 */
public String getProduct(String productId) {
    // 1. 查询缓存
    String cacheKey = "product:" + productId;
    String cached = redisTemplate.opsForValue().get(cacheKey);
    if (cached != null) {
        return "【缓存】" + cached;
    }

    // 2. 布隆过滤器拦截：如果ID不存在，直接返回空（避免查库）
    if (!bloomFilter.mightContain(productId)) {
        return "【布隆拦截】商品不存在";
    }

    // 3. 可能误判（小概率）或确实存在，查数据库
    String dbResult = queryDatabase(productId);
    if (dbResult != null) {
        // 写入缓存，设置过期时间
        redisTemplate.opsForValue().set(cacheKey, dbResult, 30, TimeUnit.MINUTES);
        return "【数据库】" + dbResult;
    } else {
        // 布隆过滤器存在但数据库没有（理论上极少发生），可以缓存空值或返回空
        return "【数据库未找到】";
    }
}
```

缓存击穿

- 定义
  - 某个热点Key在失效的瞬间，大量请求同时打穿缓存，直达数据库
- 典型场景：
  - 秒杀商品信息失效
  - 微博热搜榜更新
  - 爆款商品详情页

防御缓存击穿：分布式锁

- 分布式锁的核心思想：
  - 将并发对数据库的访问串行化，在缓存失效时，确保只有一个线程（或进程）去数据库加载数据，其他线程等待或复用结果。

```java
public Object getHotData(String key) {
    Object value = redis.get(key);
    if (value != null) return value;

    // 尝试获取分布式锁
    String lockKey = "lock:" + key;
    if (redis.setnx(lockKey, "1", 5)) { // 5秒过期
        try {
            value = db.query(key);
            redis.setex(key, value, 3600);
        } finally {
            redis.del(lockKey);
        }
    } else {
        // 其他线程等待50ms后重试（可考虑循环重试）
        Thread.sleep(50);
        return redis.get(key); // 可能已加载好
    }
    return value;
}
```

防御缓存击穿：分布式锁

- 分布式锁的注意事项
  - 锁的粒度：锁应与具体的热点 key 绑定（如 lock:product:1001），避免影响其他 key。
  - 锁超时：必须设置锁的自动过期时间（如 5 秒），防止获得锁的线程崩溃导致死锁。过期时间应略大于数据库查询+网络开销。
  - 锁重试策略：未获得锁的线程不能无限等待，通常设置重试次数或最大等待时间。
  - 锁删除：只能由持有锁的线程删除（可通过记录唯一标识实现），避免误删他人锁。

防御缓存击穿：逻辑永不过期

- 逻辑用不过期策略：
  - 缓存永远不设置物理过期时间，结合后台异步刷新，能避免热点 Key 失效瞬间的并发冲击，又能保证数据最终一致
- 实现方法：
  - 若未过期，直接返回缓存数据。
  - 若已过期，立即返回旧数据（牺牲短暂一致性），同时异步触发刷新任务，从数据库加载新数据并更新缓存中的逻辑过期时间。

方案	一致性	性能	复杂度	适用场景
分布式锁	强最终一致	有锁开销	中	对一致性要求稍高场景
逻辑永不过期	最终一致（有延迟）	极高（无锁）	较高	允许短暂不一致，追求性能

武漢大學 WUHAN UNIVERSITY

防御缓存击穿：逻辑永不过期

```java
public class CacheValue<T> {
    private T data;             // 业务数据
    private long expireTime;    // 逻辑过期时间（时间戳）
    // 构造方法、getter/setter
}
```

```java
public Object getData(String key) {
    CacheValue<Object> cacheValue = redisTemplate.opsForValue()
    if (cacheValue == null) {
        // 首次访问（缓存未命中），可以同步加载（或回源数据库）
        return loadFromDBAndSetCache(key);
    }

    // 检查逻辑是否过期
    if (cacheValue.getExpireTime() >
            System.currentTimeMillis()) {
        // 未过期：直接返回
        return cacheValue.getData();
    }
```

```java
// 已过期：返回旧数据，同时异步刷新
refreshPool.execute(() -> {
    // 分布式锁防止多个线程同时刷新同一个 key（可选）
    String lockKey = "refresh:" + key;
    if (tryLock(lockKey)) {
        try {
            // 再次检查是否已被其他线程刷新（双重检查）
            CacheValue<Object> latest = redisTemplate.opsForValue().get(key);
            if (latest != null && latest.getExpireTime() <= System.currentTimeMillis())
                Object newData = db.query(key);
                putCache(key, newData, 3600); // 重新设置缓存，更新逻辑过期时间
            }
        } finally {
            releaseLock(lockKey);
        }
    }
});

return cacheValue.getData(); // 返回旧数据
```

缓存雪崩

- 缓存雪崩
  - 大量缓存同时过期或Redis集群整体故障，导致请求全部打到数据库
- 典型场景
  - 所有key设置相同过期时间
  - 系统重启后缓存清空
  - Redis集群宕机

策略	说明	效果
过期时间加随机值	基础过期时间 + random(0-300秒)	避免集体失效
缓存预热	系统启动时预先加载热点数据	避免冷启动空窗期
多级缓存	本地缓存 + Redis，本地缓存保底	即使Redis宕机，本地缓存仍可用
限流熔断	数据库前加限流保护	防止数据库被冲垮
Redis高可用	主从+Sentinel/Cluster	避免单点故障

WUHAN UNIVERSITY

防御缓存雪崩：过期控制

- **过期时间加随机值（最常用）**
  - 避免所有 key 同时过期，缓解压力。
- **异步延迟双写机制**
  - 提前批量刷新即将过期的 key
  - 配合队列/定时任务异步更新

```
// 基础过期时间1小时，加随机0-5分钟
int baseExpire = 3600;
int randomExpire = new Random().nextInt(300);
redis.setex(key, value, baseExpire + randomExpire);
```

防御缓存雪崩：缓存预热

- **冷启动问题**
  - 系统刚启动时缓存为空，大量请求直接打穿到数据库，导致数据库瞬间过载
- **预热四步法**
  - 离线分析：根据历史日志分析热点数据
  - 分级加载：核心数据全量预加载，次重要数据按需异步加载
  - 流量控制：控制预热速度，避免预热本身压垮Redis
  - 进度监控：监控预热完成率

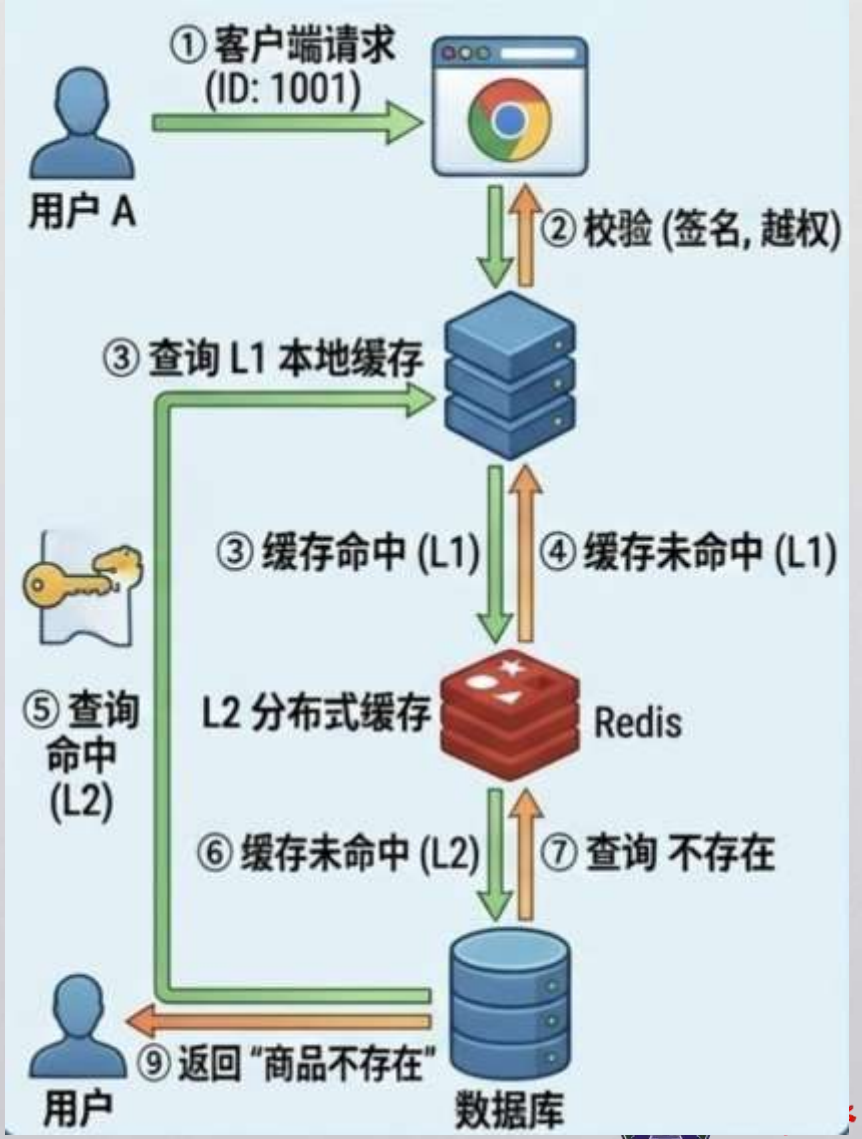
```java
@Component
public class CachePreloader implements ApplicationRunner {
    @Autowired
    private RedisTemplate redisTemplate;

    @Override
    public void run(ApplicationArguments args) {
        List<String> hotKeys = loadHotKeysFromDB();
        hotKeys.parallelStream().forEach(key -> {
            Object value = db.query(key);
            redisTemplate.opsForValue().set(key, value, 1, TimeUnit.HOURS);
        });
    }
}
```

防御缓存雪崩：多级缓存

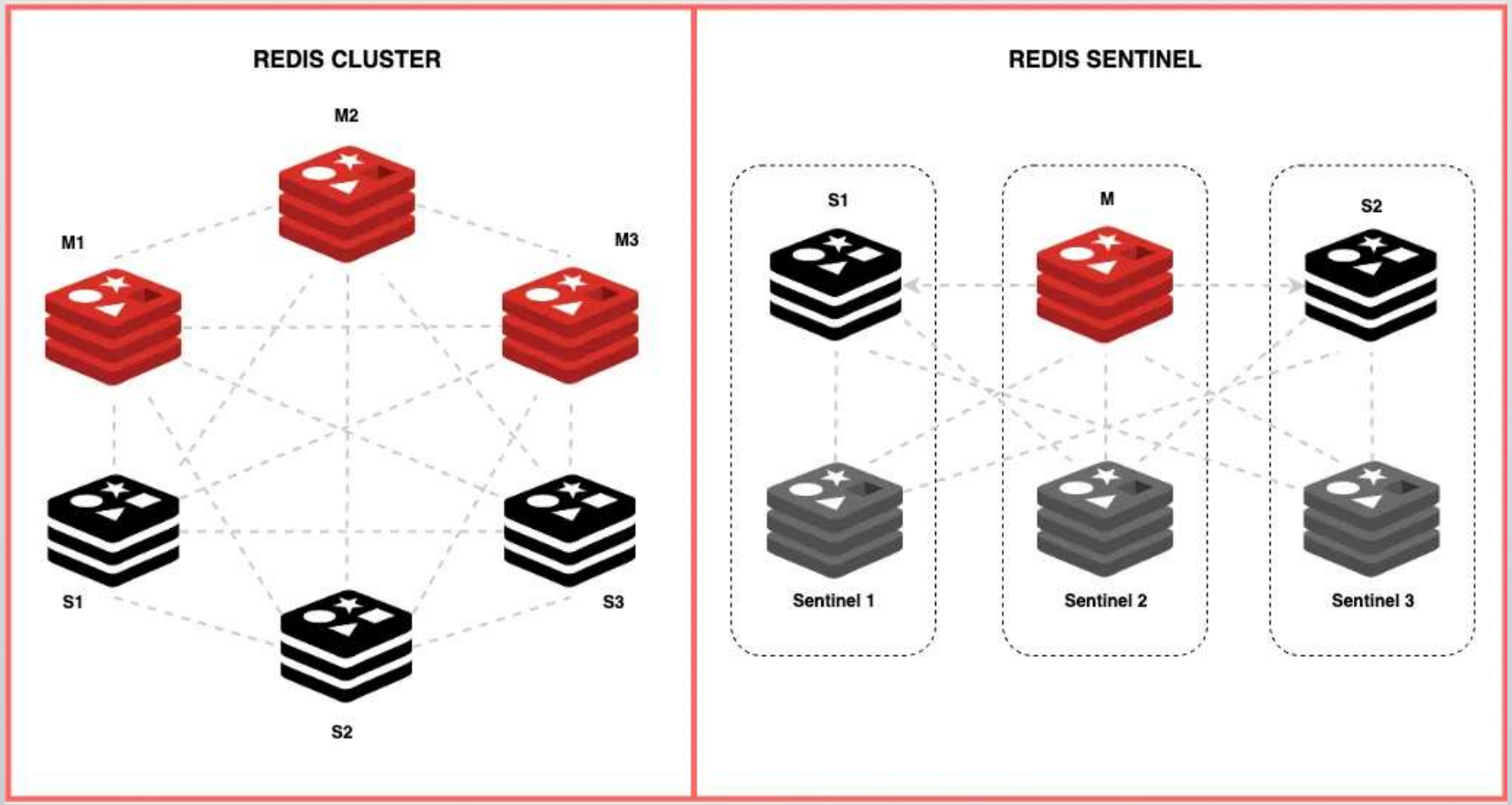
- Cloudflare实战案例
  - 本地缓存 + 数据中心级分片缓存 + 存储节点
  - 结果：缓存命中率>99.999%，90%请求<1ms

层级	存储介质	访问时间	容量	适用数据
L1 本地缓存	应用内存	纳秒级	小（MB级）	超热点、静态配置
L2 分布式缓存	Redis内存	1-3ms	大（GB/TB级）	共享数据、会话
L3 数据库	磁盘	10-50ms	海量	全量数据



防御缓存雪崩：Redis高可用部署

- Redis的三种部署模式
  - 单机模式（10万以内并发）、Redis集群（水平扩展）主从+哨兵（高可用）



武漢大學
WUHAN UNIVERSITY

作业：高并发读

- 容器环境
  - 配置项目的DockerFile和docker-compose文件，将数据库、后端服务、Ngnix分别使用容器进行启动加载。
- 负载均衡
  - 后端服务启动多个实例分别开启不同Rest端口（如8081 和 8082 端口），通过Nginx（如80端口）进行代理和转发。尝试为Nginx配置不同的负载均衡算法。
  - 使用JMeter进行压力测试。观察响应时间，并检查后端日志，验证各后端处理的请求数是否大致相等。
- 动静分离
  - 写一个简单的前端Html文件，可以包括css、js等。在Nginx中配置动静分离
  - 使用JMeter分别压测静态文件，以及后端服务，观察响应时间
- 分布式缓存
  - 引入Redis缓存，实现商品详情页缓存
  - 处理缓存穿透、击穿、雪崩问题